linear-algebra: the Linear Algebra program — user guide

linear-algebra-ai999x 1.0.3, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Linear Algebra covers vectors, matrices, determinants, linear systems, eigenvalues, decompositions, least squares, sparse matrices and numerical methods (root finding, interpolation, quadrature). This guide is about `linear-algebra`, the model as a self-contained program for Windows and Linux: 89 capabilities in 13 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/linear-algebra-ai999x/); the Python package (`pip install linear-algebra-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS; the newest version is always under https://www.ai999x.com/downloads/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Linear Algebra	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-linear-algebra-1.4.0-windows-x86_64.zip
Desktop app: Linear Algebra	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-linear-algebra-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Linear Algebra in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-linear-algebra-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-linear-algebra.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-linear-algebra-1.4.0-linux-
x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-linear-algebra-1.4.0-linux-x86_64.tar.gz
./ai999x-linear-algebra-1.4.0-linux-x86_64/ai999x-linear-algebra
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `linear-algebra` is this one. The checksum line should print OK.

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
linear-algebra --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
linear-algebra --version
```

In your browser

Open https://www.ai999x.com/models/linear-algebra-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install linear-algebra-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ linear-algebra --version
linear-algebra-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ linear-algebra ask "Find the determinant of [[2, 0, 1], [1, 3, 2], [1, 1, 2]]."
Read as: Determinant of a 3 x 3 matrix
det = 6
  by: Determinant of a 3 x 3 matrix  (rule of Sarrus: three down-diagonal products minus three up-diagonal products (exact))
  assumes: A is 3 x 3
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ linear-algebra search "root of a polynomial by newton's method"
newton_root                          Root of a polynomial by Newton's method  (numerical methods /
root finding)
secant_root                          Root of a polynomial by the secant method  (numerical methods
/ root finding)
bisection_root                       Root of a polynomial by bisection  (numerical methods / root
finding)
newton_divided_differences           Newton divided differences  (numerical methods /
interpolation)
polynomial_evaluation                Evaluate a polynomial and its derivative (Horner's method)
(numerical methods / polynomials)
interpolating_polynomial             Coefficients of the interpolating polynomial  (numerical
methods / interpolation)
midpoint_rule                        Composite midpoint rule for a polynomial  (numerical methods /
quadrature)
gauss_legendre_integral              Gauss-Legendre quadrature of a polynomial  (numerical methods
/ quadrature)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ linear-algebra list
adjugate                             Adjugate (classical adjoint) of a matrix
angle_between_vectors                Angle between two vectors
bisection_root                       Root of a polynomial by bisection
change_of_basis                      Coordinates of a vector in a basis
characteristic_polynomial            Characteristic polynomial of a matrix
cholesky                             Cholesky decomposition
clamped_cubic_spline                 Clamped cubic spline interpolation
column_space                         Column space of a matrix
… 81 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ linear-algebra info newton_root
Root of a polynomial by Newton's method
  Newton-Raphson iteration until the step is below the last bit of x
  in   coefficients polynomial coefficients, highest power first (written out in x in a question)
(vector) [1]
  in   x0       starting value [1]
  out  root     the root the iteration converges to [1]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ linear-algebra solve newton_root "coefficients=[1, 0, -2, -5]" x0=2.0
root = 2.094551482
  by: Root of a polynomial by Newton's method  (Newton-Raphson iteration until the step is below
the last bit of x)
  x0 = 2
  assumes: the iteration converges (within 200 steps) to a simple, well-conditioned root; p'(x)
never vanishes at an iterate
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.


```
$ linear-algebra solve newton_root "coefficients=[1, 0, -2, -5]" x0=2.0 --json
{"outcome":"SOLVED","message":"","value":2.0945514815423265,"unit":"","solved_for":"root","si":
[["x0",2.0]],"values":[["root",2.0945514815423265,""]],"target":"root","phrase":"the root the
iteration converges to","assumptions":["the iteration converges (within 200 steps) to a simple,
well-conditioned root","p'(x) never vanishes at an iterate"],"name":"Root of a polynomial by
Newton's method","equation":"Newton-Raphson iteration until the step is below the last bit of x"}
```


2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ linear-algebra solve newton_root "coefficients=[1, 0, -2, -5]"
NEEDS_INPUT: newton_root needs x0 (starting value). Example: solve('newton_root', coefficients='1,
0, -2, -5', x0=2).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ linear-algebra figures newton_root "coefficients=[1, 0, -2, -5]" x0=2.0 --out .
[
  {
    "slug": "newton-root-live-1",
    "title": "The root the iteration converges to as polynomial coefficients, highest power first
(w... varies",
    "caption": "The root the iteration converges to as coefficients[1] runs from 0.25 to 1.75,
every other input as you entered it (41 solutions). Marked: your coefficients[1] = 1 gives root =
2.095. Your inputs: coefficients = (list), x0 = 2.0.",
    "file": ".\\newton-root-live-1.svg"
  },
  {
    "slug": "newton-root-live-2",
    "title": "The root the iteration converges to as polynomial coefficients, highest power first
(w... varies",
    "caption": "The root the iteration converges to as coefficients[4] runs from -8.75 to -1.25,
every other input as you entered it (29 solutions). Marked: your coefficients[4] = -5 gives root =
2.095. Your inputs: coefficients = (list), x0 = 2.0.",
    "file": ".\\newton-root-live-2.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | linear-algebra batch

`cases.jsonl`:

```
{"key": "newton_root", "inputs": {"coefficients": "[1, 0, -2, -5]", "x0": 2.0}}
{"key": "qr_decomposition", "inputs": {"A": "[[3, 1], [4, 2]]"}}
```

```
$ linear-algebra batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":2.0945514815423265,"unit":"","solved_for":"root","si":
[["x0",2.0]],"values":[["root",2.0945514815423265,""]],"target":"root","phrase":"the root the
iteration converges to","assumptions":["the iteration converges (within 200 steps) to a simple,
well-conditioned root","p'(x) never vanishes at an iterate"],"name":"Root of a polynomial by
Newton's method","equation":"Newton-Raphson iteration until the step is below the last bit of x"}
{"outcome":"SOLVED","message":"","value":null,"unit":"","solved_for":"R","values":[["Q",
[[0.5999999999999999,-0.7999999999999999],[0.7999999999999999,0.6000000000000001]],""],["R",
[[5.0,2.1999999999999997],[0.0,0.4000000000000013]],""]],"target":"R","phrase":"m x n matrix with
orthonormal columns","assumptions":["rows >= columns","independent columns","at most 10 x
10","convention: the diagonal of R is positive (unique)"],"name":"QR decomposition
(Householder)","equation":"Householder reflections, then signs fixed so that diag(R) > 0"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `linear-algebra info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`linear-algebra solve ... --json` prints one answer as one line of JSON. `linear-algebra batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | linear-algebra batch`. `linear-algebra catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["linear-algebra", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`linear-algebra figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ linear-algebra figures newton_root "coefficients=[1, 0, -2, -5]" x0=2.0 --out .
[
  {
    "slug": "newton-root-live-1",
    "title": "The root the iteration converges to as polynomial coefficients, highest power first
(w... varies",
    "caption": "The root the iteration converges to as coefficients[1] runs from 0.25 to 1.75,
every other input as you entered it (41 solutions). Marked: your coefficients[1] = 1 gives root =
2.095. Your inputs: coefficients = (list), x0 = 2.0.",
    "file": ".\\newton-root-live-1.svg"
  },
  {
    "slug": "newton-root-live-2",
    "title": "The root the iteration converges to as polynomial coefficients, highest power first
(w... varies",
    "caption": "The root the iteration converges to as coefficients[4] runs from -8.75 to -1.25,
every other input as you entered it (29 solutions). Marked: your coefficients[4] = -5 gives root =
2.095. Your inputs: coefficients = (list), x0 = 2.0.",
    "file": ".\\newton-root-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `linear_algebra_ai999x.wasm` (https://www.ai999x.com/compute/rust/linear_algebra_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("linear_algebra_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `linear-algebra info <id>` shows the same inputs and outputs in the terminal.

Decompositions (6)

Best rank-k approximation (Eckart-Young) — `low_rank_approximation`

The rank-k matrix closest to A in the 2- and Frobenius norms (truncated SVD), with both approximation errors.

$$A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

symbol		meaning	SI unit
`A`	in	the matrix	—
`k`	in	target rank k	—
`approximation`	out	the best rank-k approximation A_k	—
`error2`	out	|A - A_k|_2 = sigma_(k+1)	—
`errorF`	out	|A - A_k|_F = sqrt(sigma_(k+1)^2 + ...)	—

Applies when: 1 <= k <= min(rows, columns); sigma_k > sigma_(k+1) (otherwise A_k is not unique); at most 10 x 10.

```
$ linear-algebra solve low_rank_approximation "A=[[3, 0], [0, 1]]" k=1
approximation = [[3.0,0.0],[0.0,0.0]]
error2 = 1
errorF = 1
  by: Best rank-k approximation (Eckart-Young)  (one-sided Jacobi SVD, keep the k largest singular
triplets)
  assumes: 1 <= k <= min(rows, columns); sigma_k > sigma_(k+1) (otherwise A_k is not unique); at
most 10 x 10
```

What the worked example means: This calculation produces the closest matrix of rank k to the input matrix A, denoted as [[3, 0], [0, 0]]. The method computes this by retaining only the largest k singular values and their corresponding vectors from the singular value decomposition. The resulting approximation minimizes the distance to A in both the spectral norm and the Frobenius norm. The specific errors introduced by this truncation are reported as 1 for the spectral norm and 1 for the Frobenius norm.

Cholesky decomposition — `cholesky`

Factor a symmetric positive definite matrix as A = L L^T with L lower triangular, positive diagonal.

$$A = LL^T$$

symbol		meaning	SI unit
`A`	in	a symmetric positive definite matrix	—

symbol		meaning	SI unit
`L`	out	lower triangular factor with positive diagonal	—

Applies when: A is symmetric; A is positive definite (checked exactly by its leading minors); at most 10 x 10.

```
$ linear-algebra solve cholesky "A=[[4, 12, -16], [12, 37, -43], [-16, -43, 98]]"
L = [[2.0,0.0,0.0],[6.0,1.0,0.0],[-8.0,5.0,3.0]]
  by: Cholesky decomposition  (the Cholesky-Banachiewicz recursion l_jj = sqrt(a_jj - sum l_jk^2),
l_ij = (a_ij - sum l_ik l_jk)/l_jj)
  assumes: A is symmetric; A is positive definite (checked exactly by its leading minors); at most
10 x 10
```

What the worked example means: This calculation computes the Cholesky decomposition of the symmetric positive definite matrix [4 12 -16; 12 37 -43; -16 -43 98], yielding the lower triangular factor [[2, 0, 0], [6, 1, 0], [-8, 5, 3]]. The result means that multiplying [[2, 0, 0], [6, 1, 0], [-8, 5, 3]] by its transpose reproduces the original matrix, which simplifies solving systems and computing determinants for this specific type of matrix.

LU decomposition with partial pivoting — `lu_decomposition`

Factor a nonsingular square matrix as A = P L U (permutation, unit lower triangular, upper triangular).

$$A = PLU$$

symbol		meaning	SI unit
`A`	in	a nonsingular square matrix	—
`P`	out	permutation matrix (A = P L U)	—
`L`	out	unit lower triangular factor, |l_ij| <= 1	—
`U`	out	upper triangular factor	—

Applies when: A is square and nonsingular; at most 10 x 10; pivot: the first row of largest magnitude in the column (the scipy.linalg.lu convention A = P L U).

```
$ linear-algebra solve lu_decomposition "A=[[1, 2], [3, 4]]"
P = [[0.0,1.0],[1.0,0.0]]
L = [[1.0,0.0],[0.3333333333333333,1.0]]
U = [[3.0,4.0],[0.0,0.6666666666666667]]
  by: LU decomposition with partial pivoting  (Gaussian elimination with partial pivoting
(Doolittle form) in floating point)
  assumes: A is square and nonsingular; at most 10 x 10; pivot: the first row of largest magnitude
in the column (the scipy.linalg.lu convention A = P L U)
```

What the worked example means: This calculation factors the square nonsingular matrix [1 2; 3 4] into three components: a permutation matrix [[0, 1], [1, 0]], a unit lower triangular matrix [[1, 0], [0.333333, 1]], and an upper triangular matrix [[3, 4], [0, 0.666667]]. The product of these matrices reconstructs the original input, satisfying the relationship where the permuted rows of the lower and upper factors combine to form the initial data. This decomposition is achieved through Gaussian elimination with partial pivoting, ensuring numerical stability by selecting the largest available pivot element at each step.

Polar decomposition — `polar_decomposition`

Write a nonsingular square matrix as A = U P: U orthogonal (a rotation or reflection), P symmetric positive definite (a stretch).

$$A = UP, \quad P = (A^T A)^{1/2},\ U = AP^{-1}$$

symbol		meaning	SI unit
`A`	in	a nonsingular square matrix	—
`U`	out	orthogonal factor	—
`P`	out	symmetric positive definite factor (A^T A)^(1/2)	—

Applies when: A is square and nonsingular; at most 10 x 10; right polar form A = U P (scipy.linalg.polar's default).

```
$ linear-algebra solve polar_decomposition "A=[[0, -2], [1, 0]]"
U = [[0.0,-1.0],[1.0,0.0]]
P = [[1.0,0.0],[0.0,2.0]]
  by: Polar decomposition  (from the one-sided Jacobi SVD A = W S V^T: U = W V^T, P = V S V^T)
  assumes: A is square and nonsingular; at most 10 x 10; right polar form A = U P
(scipy.linalg.polar's default)
```

What the worked example means: Polar decomposition separates a nonsingular square matrix into two distinct geometric components. For the input matrix [0 -2; 1 0], the algorithm computes an orthogonal factor [[0, -1], [1, 0]], which represents a pure rotation or reflection, and a symmetric positive definite factor [[1, 0], [0, 2]], which represents a pure stretch. The product of these factors reconstructs the original transformation, allowing you to analyze rotational and scaling effects independently.

QR decomposition (Householder) — `qr_decomposition`

Factor A = Q R with orthonormal columns Q and upper triangular R (reduced form, positive diagonal of R).

$$A = QR,\ Q^T Q = I$$

symbol		meaning	SI unit
`A`	in	a matrix with at least as many rows as columns, independent columns	—
`Q`	out	m x n matrix with orthonormal columns	—
`R`	out	n x n upper triangular, positive diagonal	—

Applies when: rows >= columns; independent columns; at most 10 x 10; convention: the diagonal of R is positive (unique).

```
$ linear-algebra solve qr_decomposition "A=[[3, 1], [4, 2]]"
Q = [[0.5999999999999999,-0.7999999999999999],[0.7999999999999999,0.6000000000000001]]
R = [[5.0,2.1999999999999997],[0.0,0.40000000000000013]]
  by: QR decomposition (Householder)  (Householder reflections, then signs fixed so that diag(R) >
0)
  assumes: rows >= columns; independent columns; at most 10 x 10; convention: the diagonal of R is
positive (unique)
```

What the worked example means: This capability computes the QR decomposition of the input matrix [3 1; 4 2] using Householder reflections. The result consists of two matrices: [[0.6, -0.8], [0.8, 0.6]], which has orthonormal columns, and [[5, 2.2], [-0, 0.4]], which is upper triangular with positive diagonal entries. This factorization expresses [3 1; 4 2] as the product of [[0.6, -0.8], [0.8, 0.6]] and [[5, 2.2], [-0, 0.4]], ensuring numerical stability and a unique representation under the specified conditions.

Singular values of a matrix — `singular_values`

The singular values of a matrix (any shape up to 10 x 10), largest first.

$$A = U\Sigma V^T, \quad \sigma_i = \sqrt{\lambda_i(A^T A)}$$

symbol		meaning	SI unit
`A`	in	the matrix	—
`singular_values`	out	singular values, descending	—

Applies when: at most 10 x 10.

```
$ linear-algebra solve singular_values "A=[[3, 0], [4, 5]]"
singular_values = [6.708203932499369,2.2360679774997894]
  by: Singular values of a matrix  (one-sided Jacobi (Hestenes) orthogonalization of the columns;
the column lengths are the singular values)
  assumes: at most 10 x 10
```

What the worked example means: This calculation determines the singular values for the input matrix A. The method uses one-sided Jacobi orthogonalization to compute these scaling factors, which represent how much the linear transformation stretches or compresses space along its principal axes. The resulting values, shown as [6.7082, 2.23607], indicate the magnitude of this distortion, with the largest value corresponding to the direction of greatest stretch.

Determinants (5)

Adjugate (classical adjoint) of a matrix — `adjugate`

adj(A): the transpose of the matrix of cofactors; A adj(A) = det(A) I.

$$\mathrm{adj}(A)_{ij} = C_{ji}$$

symbol		meaning	SI unit
`A`	in	a square matrix (up to 7 x 7)	—
`adjugate`	out	the adjugate matrix adj(A)	—

Applies when: A is square; at most 7 x 7.

```
$ linear-algebra solve adjugate "A=[[1, 2], [3, 4]]"
adjugate = [[4.0,-2.0],[-3.0,1.0]]
  by: Adjugate (classical adjoint) of a matrix  (entry (i, j) = cofactor C_ji, every minor computed
exactly)
  assumes: A is square; at most 7 x 7
```

What the worked example means: This calculation produces the adjugate matrix, which is the transpose of the cofactor matrix derived from the input square matrix [1 2; 3 4]. Each entry in the resulting matrix [[4, -2], [-3, 1]] corresponds to a specific signed minor of the original data. This result is fundamental because multiplying the original matrix by this adjugate yields a diagonal matrix scaled by the determinant, providing a direct path to finding the inverse without row reduction.

Determinant of a 2 x 2 matrix — `determinant_2x2`

det [[a, b], [c, d]] = ad - bc.

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

symbol		meaning	SI unit
`A`	in	a 2 x 2 matrix	—
`det`	out	the determinant ad - bc	—

Applies when: A is 2 x 2.

```
$ linear-algebra solve determinant_2x2 "A=[[1, 2], [3, 4]]"
det = -2
  by: Determinant of a 2 x 2 matrix  (ad - bc (exact decimal arithmetic))
  assumes: A is 2 x 2
```

What the worked example means: This calculation determines the determinant of a two-by-two matrix by multiplying the top-left entry by the bottom-right entry, then subtracting the product of the top-right and bottom-left entries. The resulting value, -2, represents the signed area scaling factor of the linear transformation defined by the matrix. A non-zero result indicates that the matrix is invertible, meaning the transformation does not collapse space into a lower dimension.

Determinant of a 3 x 3 matrix — `determinant_3x3`

The determinant of a 3 x 3 matrix by the rule of Sarrus.

$$\det A = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33}$$

symbol		meaning	SI unit
`A`	in	a 3 x 3 matrix	—
`det`	out	the determinant	—

Applies when: A is 3 x 3.

```
$ linear-algebra solve determinant_3x3 "A=[[2, 0, 1], [1, 3, 2], [1, 1, 2]]"
det = 6
  by: Determinant of a 3 x 3 matrix  (rule of Sarrus: three down-diagonal products minus three up-diagonal products (exact))
  assumes: A is 3 x 3
```

What the worked example means: This calculation finds the determinant of a three by three matrix using the rule of Sarrus. The result, 6, represents the scaling factor associated with the linear transformation defined by the input matrix A. It is computed by summing the products of the down diagonals and subtracting the products of the up diagonals.

Determinant of an n x n matrix — `determinant`

The determinant of a square matrix of any size up to 10 x 10, by row reduction.

$$\det A = (-1)^{s} \prod_{i} u_{ii}$$

symbol		meaning	SI unit
`A`	in	a square matrix (up to 10 x 10)	—
`det`	out	the determinant	—

Applies when: A is square; at most 10 x 10.

```
$ linear-algebra solve determinant "A=[[1, 2, 0, 0], [3, 4, 0, 0], [0, 0, 5, 6], [0, 0, 7, 8]]"
det = 4
  by: Determinant of an n x n matrix  (Gaussian elimination with partial pivoting in exact rational arithmetic: det = (-1)^swaps x product of pivots)
  assumes: A is square; at most 10 x 10
```

What the worked example means: This calculation determines the determinant for a square matrix with dimensions up to ten by ten. The method uses Gaussian elimination with partial pivoting in exact rational arithmetic, transforming the matrix into upper triangular form. The final result, 4, represents the scalar value that characterizes the scaling factor of the linear transformation described by the matrix.

Minor and cofactor of an entry — `minor_cofactor`

The minor M_ij (determinant with row i and column j deleted) and the cofactor C_ij = (-1)^(i+j) M_ij.

$$C_{ij} = (-1)^{i+j} M_{ij}$$

symbol		meaning	SI unit
`A`	in	a square matrix	—
`i`	in	row of the entry (1-based)	—
`j`	in	column of the entry (1-based)	—
`minor`	out	the minor M_ij	—
`cofactor`	out	the cofactor C_ij	—

Applies when: A is square, at least 2 x 2; 1 <= i, j <= n.

```
$ linear-algebra solve minor_cofactor "A=[[1, 2, 3], [4, 5, 6], [7, 8, 10]]" i=2 j=3
minor = -6
cofactor = 6
  by: Minor and cofactor of an entry  (delete row i and column j, take the determinant exactly;
attach the sign (-1)^(i+j))
  assumes: A is square, at least 2 x 2; 1 <= i, j <= n
```

What the worked example means: This calculation determines the minor and cofactor for the entry located at row 2 and column 3 of matrix A. The minor, reported as -6, is the determinant of the submatrix formed by deleting that specific row and column. The cofactor, reported as 6, adjusts this value by applying a sign based on the position indices. These values are fundamental components used in expanding determinants along rows or columns and in constructing the adjugate matrix.

Eigenvalues (8)

Characteristic polynomial of a matrix — `characteristic_polynomial`

The coefficients of det(lambda I - A) = lambda^n + c1 lambda^(n-1) + ... + cn, highest power first.

$$p(\lambda) = \det(\lambda I - A)$$

symbol		meaning	SI unit
`A`	in	a square matrix (up to 10 x 10)	—
`coefficients`	out	[1, c1, ..., cn] of det(lambda I - A), highest power first	—

Applies when: A is square; at most 10 x 10.

```
$ linear-algebra solve characteristic_polynomial "A=[[2, 1], [1, 2]]"
coefficients = [1.0,-4.0,3.0]
  by: Characteristic polynomial of a matrix  (Faddeev-LeVerrier recursion in exact rational
arithmetic)
  assumes: A is square; at most 10 x 10
```

What the worked example means: This calculation determines the coefficients of the characteristic polynomial for the square matrix [2 1; 1 2] using exact rational arithmetic. The resulting list [1, -4, 3] represents the terms of the equation det(lambda I - A) = lambda^n + c1 lambda^(n-1) + ... + cn, ordered from the highest power to the constant term. These values define the polynomial whose roots are the eigenvalues of the input matrix.

Dominant eigenvalue by power iteration — `power_iteration`

The eigenvalue of largest magnitude and its unit eigenvector, by the power method.

$$\mathbf{x}_{k+1} = \frac{A\mathbf{x}_k}{\|A\mathbf{x}_k\|}, \quad \lambda \approx \mathbf{x}_k^T A \mathbf{x}_k$$

symbol		meaning	SI unit
`A`	in	a square matrix with a single real dominant eigenvalue	—
`eigenvalue`	out	the dominant eigenvalue	—
`eigenvector`	out	its unit eigenvector (first largest entry positive)	—

Applies when: A is square, at most 10 x 10; one real eigenvalue is strictly largest in magnitude.

```
$ linear-algebra solve power_iteration "A=[[4, 1], [2, 3]]"
eigenvalue = 5
eigenvector = [0.7071067811865475,0.7071067811865475]
  by: Dominant eigenvalue by power iteration  (power iteration x <- A x / |A x| to convergence,
polished by two steps of shifted inverse iteration; the dominance is checked against the QR-
algorithm spectrum)
  assumes: A is square, at most 10 x 10; one real eigenvalue is strictly largest in magnitude
```

What the worked example means: This calculation determines the dominant eigenvalue, defined as the eigenvalue with the largest magnitude, and its corresponding unit eigenvector for the input matrix [4 1; 2 3]. The result 5 represents this extreme value, while [0.707107, 0.707107] provides the direction associated with it. This method iteratively refines an initial guess to converge on these values, ensuring the eigenvector is normalized and oriented such that its first largest entry is positive.

Eigenvalues and eigenvectors of a symmetric matrix — `symmetric_eigenvectors`

The orthonormal eigenvectors of a real symmetric matrix with distinct eigenvalues (spectral decomposition).

$$A\mathbf{q}_i = \lambda_i \mathbf{q}_i,\ Q^T Q = I$$

symbol		meaning	SI unit
`A`	in	a symmetric matrix with distinct eigenvalues	—
`eigenvalues`	out	eigenvalues, ascending	—
`eigenvectors`	out	unit eigenvectors in the same order, one per row	—

Applies when: A is symmetric; the eigenvalues are distinct; at most 10 x 10; sign convention: the first entry of largest magnitude of each eigenvector is positive.

```
$ linear-algebra solve symmetric_eigenvectors "A=[[2, 1], [1, 2]]"
eigenvalues = [0.9999999999999998,2.9999999999999996]
eigenvectors = [[0.7071067811865476,-0.7071067811865476],[0.7071067811865476,0.7071067811865476]]
  by: Eigenvalues and eigenvectors of a symmetric matrix  (cyclic Jacobi rotations; the accumulated
rotations are the eigenvectors)
  assumes: A is symmetric; the eigenvalues are distinct; at most 10 x 10; sign convention: the
first entry of largest magnitude of each eigenvector is positive
```

What the worked example means: This calculation determines the spectral decomposition of a real symmetric matrix by applying cyclic Jacobi rotations to diagonalize it. The resulting values, [1, 3], represent the scaling factors along principal axes, while [[0.707107, -0.707107], [0.707107, 0.707107]] provide the directions of these axes as unit vectors arranged in ascending order of their associated eigenvalues.

Eigenvalues of a 2 x 2 matrix — `eigenvalues_2x2`

Both eigenvalues of a 2 x 2 matrix from its characteristic equation lambda^2 - tr(A) lambda + det(A) = 0.

$$\lambda = \frac{\operatorname{tr} A \pm \sqrt{(\operatorname{tr} A)^2 - 4 \det A}}{2}$$

symbol		meaning	SI unit
`A`	in	a 2 x 2 matrix	—
`lambda1`	out	first eigenvalue (the smaller real one, or the one with positive imaginary part)	—
`lambda2`	out	second eigenvalue	—

Applies when: A is 2 x 2.

```
$ linear-algebra solve eigenvalues_2x2 "A=[[2, 1], [1, 2]]"
lambda1 = 1
lambda2 = 3
  by: Eigenvalues of a 2 x 2 matrix  (quadratic formula on the characteristic polynomial (trace and
determinant exact, stable form))
  assumes: A is 2 x 2
```

What the worked example means: This calculation determines the two eigenvalues for the input 2x2 matrix A. Eigenvalues represent the scalar factors by which specific vectors, known as eigenvectors, are stretched or compressed when the linear transformation defined by the matrix is applied. Solving the characteristic equation using the trace and determinant shows that the resulting values are 1 and 3. These numbers describe the fundamental scaling behavior of the system represented by the matrix.

Eigenvalues of a general square matrix (QR algorithm) — `general_eigenvalues`

All eigenvalues (real or complex) of a real square matrix, and its spectral radius.

$$\det(A - \lambda I) = 0, \quad \rho(A) = \max_i |\lambda_i|$$

symbol		meaning	SI unit
`A`	in	a square matrix (up to 10 x 10)	—
`eigenvalues`	out	eigenvalues: real ones ascending, then complex pairs by real part, positive imaginary part first	—
`spectral_radius`	out	largest |eigenvalue|	—

Applies when: A is square; at most 10 x 10.

```
$ linear-algebra solve general_eigenvalues "A=[[1, 2, 0], [0, 3, 0], [2, -4, 2]]"
eigenvalues = [0.999999999999998,1.9999999999999991,3.0000000000000018]
spectral_radius = 3
  by: Eigenvalues of a general square matrix (QR algorithm)  (Householder reduction to Hessenberg
form, then the shifted QR algorithm (Wilkinson shifts, Givens rotations, deflation))
  assumes: A is square; at most 10 x 10
```

What the worked example means: This calculation determines all eigenvalues, both real and complex, for the square matrix [1 2 0; 0 3 0; 2 -4 2]. The process involves reducing the matrix to Hessenberg form using Householder transformations and then applying the shifted QR algorithm with Wilkinson shifts to find the roots of the characteristic equation. The resulting values in [1, 2, 3] represent the scaling factors associated with the eigenvectors, while 3 indicates the magnitude of the largest eigenvalue, which is crucial for assessing system stability.

Eigenvalues of a symmetric matrix (Jacobi method) — `symmetric_eigenvalues`

All eigenvalues of a real symmetric matrix (they are real), in ascending order.

$$A = Q \Lambda Q^T$$

symbol		meaning	SI unit
`A`	in	a symmetric matrix (up to 10 x 10)	—
`eigenvalues`	out	eigenvalues, ascending	—

Applies when: A is symmetric; at most 10 x 10.

```
$ linear-algebra solve symmetric_eigenvalues "A=[[2, 1, 0], [1, 2, 1], [0, 1, 2]]"
eigenvalues = [0.585786437626905,1.999999999999999,3.4142135623730936]
  by: Eigenvalues of a symmetric matrix (Jacobi method)  (cyclic Jacobi rotations until the off-diagonal part vanishes)
  assumes: A is symmetric; at most 10 x 10
```

What the worked example means: This calculation determines all eigenvalues for the input symmetric matrix [2 1 0; 1 2 1; 0 1 2]. Because the matrix is real and symmetric, the resulting values in [0.585786, 2, 3.41421] are guaranteed to be real numbers. These eigenvalues represent the principal scaling factors along the matrix's eigenvector directions, providing essential insight into the linear transformation's spectral properties.

Positive definiteness test (Sylvester's criterion) — `positive_definiteness`

Whether a symmetric matrix is positive definite, with its leading principal minors.

$$A \succ 0 \iff D_k = \det A_{[1..k],[1..k]} > 0,\ k = 1..n$$

symbol		meaning	SI unit
`A`	in	a symmetric matrix	—
`positive_definite`	out	true when x^T A x > 0 for every x != 0	—
`leading_minors`	out	the leading principal minors D1, ..., Dn	—

Applies when: A is symmetric; at most 10 x 10.

```
$ linear-algebra solve positive_definiteness "A=[[2, -1], [-1, 2]]"
positive_definite = true
leading_minors = [2.0,3.0]
  by: Positive definiteness test (Sylvester's criterion)  (Sylvester's criterion: positive definite
exactly when every leading principal minor is positive (exact determinants))
  assumes: A is symmetric; at most 10 x 10
```

What the worked example means: This calculation determines whether the symmetric matrix A is positive definite by applying Sylvester's criterion. It computes the sequence of leading principal minors, reported here as [2, 3]. If all these determinants are positive, the boolean output yes is true, indicating that the quadratic form x transpose A x is always positive for non-zero vectors. This property ensures the matrix has positive eigenvalues and defines a convex function with a unique minimum.

Smallest eigenvalue by inverse iteration — `inverse_iteration`

The eigenvalue of smallest magnitude and its unit eigenvector, by inverse power iteration.

$$A\mathbf{y}_{k+1} = \mathbf{x}_k, \quad \mathbf{x}_{k+1} = \mathbf{y}_{k+1} / \|\mathbf{y}_{k+1}\|$$

symbol		meaning	SI unit
`A`	in	a nonsingular square matrix with a single real eigenvalue of smallest magnitude	—
`eigenvalue`	out	the eigenvalue of smallest magnitude	—
`eigenvector`	out	its unit eigenvector (first largest entry positive)	—

Applies when: A is square and nonsingular, at most 10 x 10; one real eigenvalue is strictly smallest in magnitude.

```
$ linear-algebra solve inverse_iteration "A=[[4, 1], [2, 3]]"
eigenvalue = 2
eigenvector = [-0.447213595499958,0.894427190999916]
  by: Smallest eigenvalue by inverse iteration  (power iteration with A^-1 (one LU factorization, a
solve per step), polished by shifted inverse iteration; checked against the QR-algorithm spectrum)
  assumes: A is square and nonsingular, at most 10 x 10; one real eigenvalue is strictly smallest
in magnitude
```

What the worked example means: This calculation determines the eigenvalue of smallest magnitude for matrix [4 1; 2 3] using inverse power iteration. The method solves a linear system at each step to converge on the dominant mode of the inverse matrix, which corresponds to the smallest eigenvalue of the original matrix. The resulting value 2 represents this minimum magnitude, while [-0.447214, 0.894427] provides the associated direction normalized to unit length.

Least squares (5)

Least-squares polynomial fit — `least_squares_polynomial`

The polynomial c0 + c1 x + ... + cd x^d of a given degree that best fits data in the least-squares sense.

$$\min_{\mathbf{c}} \sum_{i} \left(y_i - \sum_{k=0}^{d} c_k x_i^k \right)^2$$

symbol		meaning	SI unit
`x`	in	x values	—
`y`	in	y values	—
`degree`	in	degree d of the polynomial (0 to 8)	—
`coefficients`	out	[c0, c1, ..., cd], lowest power first	—

Applies when: x and y have the same number of values; more distinct x values than the degree; degree at most 8.

```
$ linear-algebra solve least_squares_polynomial "x=[0, 1, 2, 3]" "y=[1, 0, 3, 10]" degree=2
coefficients = [1.0,-3.0,2.0]
  by: Least-squares polynomial fit  (normal equations V^T V c = V^T y of the Vandermonde matrix,
solved in exact rational arithmetic)
  assumes: x and y have the same number of values; more distinct x values than the degree; degree
at most 8
```

What the worked example means: This calculation determines the polynomial curve that best fits the provided data points by minimizing the sum of squared vertical distances between the observed values and the model predictions. The result is a set of coefficients [1, -3, 2], ordered from the constant term to the highest power, which define the function for any input x within the range of [0, 1, 2, 3]. These coefficients are derived using normal equations on the Vandermonde matrix constructed from the input pairs [0, 1, 2, 3] and [1, 0, 3, 10], ensuring an exact rational solution for the specified degree 2.

Least-squares regression line — `least_squares_line`

The straight line y = a + b x that best fits data points in the least-squares sense, with R^2.

$$b = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad a = \bar{y} - b\bar{x}$$

symbol		meaning	SI unit
`x`	in	x values	—
`y`	in	y values	—
`slope`	out	slope b	—
`intercept`	out	intercept a	—
`r_squared`	out	coefficient of determination R^2	—

Applies when: x and y have the same number of values, at least 2; the x values are not all equal; the y values are not all equal (otherwise R^2 is undefined).

```
$ linear-algebra solve least_squares_line "x=[1, 2, 3, 4]" "y=[2, 3, 5, 4]"
slope = 0.8
intercept = 1.5
r_squared = 0.64
  by: Least-squares regression line  (b = Sxy / Sxx, a = mean(y) - b mean(x), R^2 = Sxy^2 / (Sxx
Syy), exact sums)
  assumes: x and y have the same number of values, at least 2; the x values are not all equal; the
y values are not all equal (otherwise R^2 is undefined)
```

What the worked example means: This calculation determines the straight line that best fits the provided data points by minimizing the sum of squared vertical errors. The resulting slope, 0.8, indicates the rate of change in the dependent variable for each unit increase in the independent variable, while the intercept, 1.5, represents the predicted value when the independent variable is zero. The coefficient of determination, 0.64, quantifies the proportion of variance in the dependent variable that is predictable from the independent variable.

Least-squares solution of an overdetermined system — `least_squares_solution`

The x minimizing |Ax - b| for a tall matrix A with independent columns, and the residual length.

$$A^T A\,\hat{\mathbf{x}} = A^T \mathbf{b}$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	least-squares solution x	—
`residual_norm`	out	|Ax - b| at the solution	—

Applies when: at least as many rows as columns; the columns of A are linearly independent; at most 10 x 10.

```
$ linear-algebra solve least_squares_solution "A=[[1, 0], [0, 1], [1, 1]]" "b=[1, 1, 0]"
solution = [0.3333333333333333,0.3333333333333333]
residual_norm = 1.154700538
  by: Least-squares solution of an overdetermined system  (normal equations A^T A x = A^T b in
exact rational arithmetic)
  assumes: at least as many rows as columns; the columns of A are linearly independent; at most 10
x 10
```

What the worked example means: This calculation determines the vector that best fits an overdetermined system by minimizing the Euclidean distance between the model prediction and the observed data. Using the normal equations, the method computes the optimal parameters as [0.333333, 0.333333], which represents the coefficients that make the error vector shortest possible. The quality of this fit is quantified by the residual norm 1.1547, indicating the magnitude of the discrepancy that remains after applying the solution.

Moore-Penrose pseudoinverse (full rank) — `pseudoinverse`

The pseudoinverse A+ of a matrix with independent columns, (A^T A)^-1 A^T, or independent rows, A^T (A A^T)^-1.

$$A^{+} = (A^{T}A)^{-1}A^{T} \text{ or } A^{T}(AA^{T})^{-1}$$

symbol		meaning	SI unit
`A`	in	a matrix of full rank (up to 10 x 10)	—
`pseudoinverse`	out	the matrix A+	—

Applies when: A has full rank (independent rows or independent columns); at most 10 x 10.

```
$ linear-algebra solve pseudoinverse "A=[[1, 0], [0, 1], [1, 1]]"
pseudoinverse = [[0.6666666666666666,-0.3333333333333333,0.3333333333333333],
[-0.3333333333333333,0.6666666666666666,0.3333333333333333]]
  by: Moore-Penrose pseudoinverse (full rank)  (the full-rank formulas in exact rational
arithmetic)
  assumes: A has full rank (independent rows or independent columns); at most 10 x 10
```

What the worked example means: The Moore-Penrose pseudoinverse [[0.666667, -0.333333, 0.333333], [-0.333333, 0.666667, 0.333333]] is derived from the input matrix [1 0; 0 1; 1 1] using exact rational arithmetic. Since [1 0; 0 1; 1 1] has full rank, the calculation applies the appropriate formula based on whether its columns or rows are independent, yielding a generalized inverse that facilitates solving least-squares problems and finding minimum-norm solutions where a standard inverse does not exist.

Orthogonal projection matrix onto a column space — `projection_matrix`

The matrix P = A (A^T A)^-1 A^T that projects orthogonally onto the column space of A.

$$P = A(A^{T}A)^{-1}A^{T}$$

symbol		meaning	SI unit
`A`	in	a matrix with independent columns	—
`projection`	out	the projection matrix P	—

Applies when: the columns of A are linearly independent; at most 10 x 10.

```
$ linear-algebra solve projection_matrix "A=[[1], [2]]"
projection = [[0.2,0.4],[0.4,0.8]]
  by: Orthogonal projection matrix onto a column space  (P = A (A^T A)^-1 A^T in exact rational
arithmetic)
  assumes: the columns of A are linearly independent; at most 10 x 10
```

What the worked example means: The calculation produces the orthogonal projection matrix [[0.2, 0.4], [0.4, 0.8]] for the input matrix [1; 2]. This matrix projects any vector in the ambient space onto the subspace spanned by the columns of [1; 2], effectively isolating the component of a vector that lies within this specific linear model. The result is derived using exact rational arithmetic to ensure precision, assuming the columns of [1; 2] are linearly independent and the matrix dimensions do not exceed ten by ten.

Linear systems (9)

Conjugate gradient method for Ax = b — `conjugate_gradient`

Solve Ax = b for a symmetric positive definite A by the method of conjugate gradients.

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k},\ \mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,\ \mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k} \mathbf{p}_k$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	the solution x	—

Applies when: A is symmetric positive definite (checked exactly); at most 10 x 10.

```
$ linear-algebra solve conjugate_gradient "A=[[4, 1], [1, 3]]" "b=[1, 2]"
solution = [0.09090909090909091,0.6363636363636364]
  by: Conjugate gradient method for Ax = b  (conjugate gradients from x = 0 (at most n steps in
exact arithmetic), restarted twice to remove rounding errors)
  assumes: A is symmetric positive definite (checked exactly); at most 10 x 10
```

What the worked example means: This process determines the vector x that satisfies the equation Ax equals b, where A is the symmetric positive definite coefficient matrix and b is the right-hand side vector. The method iteratively refines an initial guess to find this specific solution, which is reported as [0.0909091, 0.636364]. This result represents the precise coordinates that make the linear system true under the given constraints.

Gauss-Seidel iteration for Ax = b — `gauss_seidel`

Solve Ax = b by the Gauss-Seidel method (each new value used at once), iterated to convergence.

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right)$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	the converged solution x	—

Applies when: A is square and strictly diagonally dominant, or symmetric positive definite (guarantees convergence); at most 10 x 10.

```
$ linear-algebra solve gauss_seidel "A=[[4, 1], [2, 5]]" "b=[6, 12]"
solution = [1.0,2.0]
  by: Gauss-Seidel iteration for Ax = b  (x_i <- (b_i - sum_{j<i} a_ij x_j(new) - sum_{j>i} a_ij
x_j(old)) / a_ii, from x = 0, until the iterates stop changing)
  assumes: A is square and strictly diagonally dominant, or symmetric positive definite (guarantees
convergence); at most 10 x 10
```

What the worked example means: This process computes the solution vector x for the linear system Ax = b using the Gauss-Seidel method. Starting from an initial guess of zero, each component of the solution is

updated sequentially by substituting the most recently calculated values into the equation for that component. The iteration continues until the change between successive approximations falls below a specified tolerance, yielding the final result [1, 2]. This approach leverages new information immediately, often converging faster than methods that use only previous iteration values.

Jacobi iteration for Ax = b — `jacobi_iteration`

Solve Ax = b for a strictly diagonally dominant A by the Jacobi method, iterated to convergence.

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}\right)$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	the converged solution x	—

Applies when: A is square and strictly diagonally dominant by rows (guarantees convergence); at most 10 x 10.

```
$ linear-algebra solve jacobi_iteration "A=[[4, 1], [2, 5]]" "b=[6, 12]"
solution = [1.0,2.0]
  by: Jacobi iteration for Ax = b  (x_i <- (b_i - sum_{j != i} a_ij x_j) / a_ii for all i at once,
from x = 0, until the iterates stop changing)
  assumes: A is square and strictly diagonally dominant by rows (guarantees convergence); at most
10 x 10
```

What the worked example means: This calculation solves the linear system Ax equals b using the Jacobi iterative method. Starting from an initial guess of zero, each component of the solution vector is updated simultaneously based on the previous iteration's values and the right-hand side vector [6, 12]. The process repeats until the changes between iterations become negligible, yielding the final approximate solution [1, 2] for the coefficient matrix [4 1; 2 5].

Number of solutions of a linear system — `linear_system_classification`

Whether Ax = b has a unique solution, infinitely many, or none (Rouche-Capelli), with the two ranks.

$$\operatorname{rank} A < \operatorname{rank}[A \mid \mathbf{b}] \Rightarrow \text{none};\ \ \operatorname{rank} A = \operatorname{rank}[A \mid \mathbf{b}] = n \Rightarrow \text{unique}$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solutions`	out	unique / infinitely many / none	—
`rank_A`	out	rank of A	—
`rank_augmented`	out	rank of the augmented matrix [A | b]	—

Applies when: b has one entry per row of A; at most 10 x 10.

```
$ linear-algebra solve linear_system_classification "A=[[1, 1], [2, 2]]" "b=[2, 5]"
solutions = none
rank_A = 1
rank_augmented = 2
  by: Number of solutions of a linear system  (compare rank A, rank [A | b] and the number of
unknowns (exact row reduction))
  assumes: b has one entry per row of A; at most 10 x 10
```

What the worked example means: This calculation determines the nature of the solution set for a linear system by applying the Rouché-Capelli theorem. It computes the rank of the coefficient matrix and the rank of the augmented matrix formed by appending the right-hand side vector to the coefficients. If these ranks differ, as indicated by 1 versus 2, the system is inconsistent and has no solution. If they are equal, the number of unknowns dictates whether the solution is unique or part of an infinite family. The final classification provided here is none.

Reduced row echelon form — `rref`

The reduced row echelon form (RREF) of a matrix, its pivot columns (1-based) and its rank.

$$A \sim \mathrm{rref}(A)$$

symbol		meaning	SI unit
`A`	in	the matrix (up to 10 x 10)	—
`rref`	out	the reduced row echelon form	—
`pivot_columns`	out	pivot columns, 1-based	—
`rank`	out	number of pivots	—

Applies when: at most 10 x 10.

```
$ linear-algebra solve rref "A=[[1, 2, 3], [4, 5, 6], [7, 8, 9]]"
rref = [[1.0,0.0,-1.0],[0.0,1.0,2.0],[0.0,0.0,0.0]]
pivot_columns = [1.0,2.0]
rank = 2
  by: Reduced row echelon form  (Gauss-Jordan elimination in exact rational arithmetic (the RREF is
unique))
  assumes: at most 10 x 10
```

What the worked example means: This calculation determines the unique reduced row echelon form of the input matrix, denoted as [1 2 3; 4 5 6; 7 8 9]. Using Gauss-Jordan elimination with exact rational arithmetic, the process transforms the matrix into a canonical structure where leading entries are one and all other entries in their columns are zero. The result includes the transformed matrix [[1, 0, -1], [0, 1, 2], [0, 0, 0]], the specific column indices containing these leading ones [1, 2], and the total count of such pivot positions, known as the rank 2. This output provides a standardized representation that reveals the linear independence of the rows and the dimensionality of the solution space.

Solve a square linear system Ax = b — `solve_linear_system`

The unique solution of Ax = b for a nonsingular n x n matrix A (n up to 10).

$$A\mathbf{x} = \mathbf{b}$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	the solution vector x	—

Applies when: A is square and nonsingular; b has one entry per row of A; at most 10 x 10.

```
$ linear-algebra solve solve_linear_system "A=[[2, 1], [1, 3]]" "b=[3, 5]"
solution = [0.8,1.4]
  by: Solve a square linear system Ax = b  (Gaussian elimination with partial pivoting and back
substitution, in exact rational arithmetic)
  assumes: A is square and nonsingular; b has one entry per row of A; at most 10 x 10
```

What the worked example means: This calculation determines the unique solution vector [0.8, 1.4] for a square linear system defined by the coefficient matrix [2 1; 1 3] and the right-hand side vector [3, 5]. The method uses Gaussian elimination with partial pivoting in exact rational arithmetic to ensure precision. The result represents the values that satisfy the equation, meaning when you multiply the matrix [2 1; 1 3] by the solution [0.8, 1.4], you recover the original vector [3, 5]. This approach is reliable for nonsingular matrices up to ten rows and columns.

Three linear equations in three unknowns — `solve_3x3_system`

Solve three linear equations in x, y, z by Cramer's rule.

$$x_j = \frac{\det A_j}{\det A}$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	the solution (x, y, z)	—
`det`	out	determinant of the coefficient matrix	—

Applies when: A is 3 x 3; det A != 0 (otherwise there is no unique solution).

```
$ linear-algebra solve solve_3x3_system "A=[[1, 1, 1], [2, -1, 1], [1, 2, -1]]" "b=[6, 3, 2]"
solution = [1.0,2.0,3.0]
det = 7
  by: Three linear equations in three unknowns  (Cramer's rule x_j = det(A_j)/det(A) in exact
decimal arithmetic)
  assumes: A is 3 x 3; det A != 0 (otherwise there is no unique solution)
```

What the worked example means: This method computes the unique solution for three variables using Cramer's rule. It evaluates the determinant of the coefficient matrix and replaces each column with the right-hand side vector to find specific determinants. The final values are obtained by dividing these modified determinants by the main determinant, yielding [1, 2, 3] when 7 is non-zero.

Tridiagonal system (Thomas algorithm) — `tridiagonal_solve`

Solve a tridiagonal system given its three diagonals and right-hand side, in O(n) operations.

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$$

symbol		meaning	SI unit
`sub`	in	sub-diagonal a_2..a_n (n-1 entries)	—
`diag`	in	main diagonal b_1..b_n	—
`sup`	in	super-diagonal c_1..c_(n-1) (n-1 entries)	—
`d`	in	right-hand side (n entries)	—
`solution`	out	the solution x	—

Applies when: n >= 2, at most 200; no zero pivot arises in the elimination (true for diagonally dominant matrices).

```
$ linear-algebra solve tridiagonal_solve "sub=[1, 1]" "diag=[4, 4, 4]" "sup=[1, 1]" "d=[5, 6, 5]"
solution = [1.0,1.0,1.0]
  by: Tridiagonal system (Thomas algorithm)  (the Thomas algorithm (forward sweep, back
substitution) in exact rational arithmetic)
  assumes: n >= 2, at most 200; no zero pivot arises in the elimination (true for diagonally
dominant matrices)
```

What the worked example means: This calculation applies the Thomas algorithm to solve a linear system where non-zero coefficients appear only on the main diagonal and the two adjacent diagonals. Using the input vectors for the sub-diagonal, main diagonal, super-diagonal, and right-hand side, the method performs a forward sweep to eliminate variables followed by back substitution. The resulting output, [1, 1, 1], represents the unique values for each variable in the system that satisfy all equations simultaneously under exact rational arithmetic.

Two linear equations in two unknowns — `solve_2x2_system`

Solve a x + b y = e, c x + d y = f for (x, y) by Cramer's rule.

$$x = \frac{\begin{vmatrix} e & b \\ f & d \end{vmatrix}}{\begin{vmatrix} a & b \\ c & d \end{vmatrix}}, \quad y = \frac{\begin{vmatrix} a & e \\ c & f \end{vmatrix}}{\begin{vmatrix} a & b \\ c & d \end{vmatrix}}$$

symbol		meaning	SI unit
`A`	in	coefficient matrix	—
`b`	in	right-hand side	—
`solution`	out	the solution (x, y)	—
`det`	out	determinant of the coefficient matrix	—

Applies when: A is 2 x 2; det A != 0 (otherwise there is no unique solution).

```
$ linear-algebra solve solve_2x2_system "A=[[2, 3], [1, -1]]" "b=[7, 1]"
solution = [2.0,1.0]
det = -5
  by: Two linear equations in two unknowns  (Cramer's rule x_j = det(A_j)/det(A) in exact decimal
arithmetic)
  assumes: A is 2 x 2; det A != 0 (otherwise there is no unique solution)
```

What the worked example means: Cramer's rule computes the unique solution to two linear equations by evaluating ratios of determinants. The determinant of the coefficient matrix, shown as -5, must be non-zero for this method to yield a valid result. The value of x is found by replacing the first column with the constants, while y uses the second column, ensuring the system ax plus by equals e and cx plus dy equals f is satisfied.

Linear transformations (2)

Matrix times vector (image under a linear map) — `matrix_vector_product`

The vector Ax: the image of x under the linear transformation with matrix A.

$$(A\mathbf{x})_i = \sum_j a_{ij} x_j$$

symbol		meaning	SI unit
`A`	in	the matrix	—
`x`	in	the vector	—
`image`	out	the vector Ax	—

Applies when: columns of A = length of x.

```
$ linear-algebra solve matrix_vector_product "A=[[1, 2], [3, 4]]" "x=[5, 6]"
image = [17.0,39.0]
  by: Matrix times vector (image under a linear map)  (dot product of each row with x (exact
decimal arithmetic))
  assumes: columns of A = length of x
```

What the worked example means: This calculation determines the resulting vector [17, 39] by applying the linear transformation represented by matrix [1 2; 3 4] to the input vector [5, 6]. Each entry in the output vector is formed by taking the dot product of a corresponding row from [1 2; 3 4] with the entire vector [5, 6]. Geometrically, this maps the position or state described by [5, 6] into a new coordinate system or dimension defined by the columns of [1 2; 3 4], effectively showing where the original vector lands after the transformation.

Rotation of a plane vector — `rotate_vector_2d`

Rotate a 2-D vector counterclockwise about the origin by an angle given in degrees.

$$R(\theta)\mathbf{v} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \mathbf{v}$$

symbol		meaning	SI unit
`v`	in	the vector (2 entries)	—
`angle`	in	rotation angle in degrees (counterclockwise)	—
`rotated`	out	the rotated vector R(angle) v	—

Applies when: v has 2 entries; a positive angle turns counterclockwise.

```
$ linear-algebra solve rotate_vector_2d "v=[1, 0]" angle=90
rotated = [0.0,1.0]
  by: Rotation of a plane vector  (multiply by the rotation matrix [[cos t, -sin t], [sin t, cos
t]] (exact at multiples of 90 degrees))
  angle = 90
  assumes: v has 2 entries; a positive angle turns counterclockwise
```

What the worked example means: This calculation applies a rotation matrix to the input vector [1, 0], turning it counterclockwise about the origin by an angle of 90 degrees. The resulting vector [0, 1] maintains the same length as the original but points in a new direction determined by the specified angle.

Matrices (12)

Condition number of a matrix — `condition_number`

How sensitive Ax = b is to errors: the 2-norm condition number sigma_max/sigma_min and the 1-norm one.

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}, \quad \kappa_1(A) = \|A\|_1 \|A^{-1}\|_1$$

symbol		meaning	SI unit
`A`	in	a nonsingular square matrix	—
`cond2`	out	2-norm condition number sigma_max / sigma_min	—
`cond1`	out	1-norm condition number |A|_1 |A^-1|_1	—

Applies when: A is square and nonsingular; at most 10 x 10.

```
$ linear-algebra solve condition_number "A=[[2, 0], [0, 0.5]]"
cond2 = 4
cond1 = 4
  by: Condition number of a matrix  (singular values by one-sided Jacobi; the 1-norm one from the
exact inverse)
  assumes: A is square and nonsingular; at most 10 x 10
```

What the worked example means: This calculation determines the sensitivity of a linear system by computing two distinct condition numbers for the matrix. The two-norm value, 4, is derived from the ratio of the largest to smallest singular values, while the one-norm value, 4, uses the product of the matrix's one-norm and its inverse's one-norm. These figures act as multipliers for potential error growth; a higher value suggests that small perturbations in the data or rounding errors during computation can lead to large changes in the solution vector.

Difference of two matrices — `matrix_subtraction`

Subtract the second matrix from the first, entry by entry.

$$(A - B)_{ij} = a_{ij} - b_{ij}$$

symbol		meaning	SI unit
`A`	in	first matrix	—
`B`	in	matrix subtracted	—
`difference`	out	the matrix A - B	—

Applies when: A and B have the same shape.

```
$ linear-algebra solve matrix_subtraction "A=[[5, 6], [7, 8]]" "B=[[1, 1], [1, 1]]"
difference = [[4.0,5.0],[6.0,7.0]]
  by: Difference of two matrices  (entry-by-entry subtraction (exact decimal arithmetic))
  assumes: A and B have the same shape
```

What the worked example means: This calculation finds the entry-by-entry difference between two matrices. Subtracting each element of the second matrix from the corresponding element of the first gives a new matrix where every position holds the result of that subtraction. For example, the top-left entry of the resulting difference matrix [[4, 5], [6, 7]] comes from subtracting the top-left entry of [1 1; 1 1] from the top-left entry of [5 6; 7 8]. This operation requires both input matrices to have identical dimensions so that every element in one has a matching partner in the other.

Inverse of a square matrix — `matrix_inverse`

The inverse A^-1 of a nonsingular square matrix (any size up to 10 x 10).

$$AA^{-1} = A^{-1}A = I$$

symbol		meaning	SI unit
`A`	in	a nonsingular square matrix	—
`inverse`	out	the matrix A^-1	—

Applies when: A is square; det A != 0 (a singular matrix has no inverse); at most 10 x 10.

```
$ linear-algebra solve matrix_inverse "A=[[4, 7], [2, 6]]"
inverse = [[0.6,-0.7],[-0.2,0.4]]
  by: Inverse of a square matrix  (Gauss-Jordan elimination on [A | I] in exact rational
arithmetic)
  assumes: A is square; det A != 0 (a singular matrix has no inverse); at most 10 x 10
```

What the worked example means: This calculation determines the inverse of a given square matrix using Gauss-Jordan elimination on an augmented system. The input matrix [4 7; 2 6] must be nonsingular, meaning its determinant is not zero, to ensure an inverse exists. The resulting output [[0.6, -0.7], [-0.2, 0.4]] represents the unique matrix that, when multiplied by the original, yields the identity matrix. This process relies on exact rational arithmetic to maintain precision and avoid numerical instability associated with floating-point operations.

Kronecker (tensor) product of two matrices — `kronecker_product`

The block matrix A (x) B whose (i, j) block is a_ij B.

$$A \otimes B = [a_{ij}B]$$

symbol		meaning	SI unit
`A`	in	first matrix	—
`B`	in	second matrix	—
`kron`	out	the Kronecker product A (x) B	—

Applies when: each matrix at most 6 x 6.

```
$ linear-algebra solve kronecker_product "A=[[1, 2], [3, 4]]" "B=[[0, 1], [1, 0]]"
kron = [[0.0,1.0,0.0,2.0],[1.0,0.0,2.0,0.0],[0.0,3.0,0.0,4.0],[3.0,0.0,4.0,0.0]]
  by: Kronecker (tensor) product of two matrices  (block (i, j) = a_ij B (exact))
  assumes: each matrix at most 6 x 6
```

What the worked example means: This calculation forms a larger block matrix by multiplying every entry of the first matrix A, shown as [1 2; 3 4], by the entire second matrix B, shown as [0 1; 1 0]. The resulting structure, [[0, 1, 0, 2], [1, 0, 2, 0], [0, 3, 0, 4], [3, 0, 4, 0]], places scaled copies of matrix B in each position corresponding to an element of matrix A. This method creates a tensor space representation useful for analyzing combined systems where the total dimension is the product of the individual dimensions.

Norms of a matrix — `matrix_norms`

The Frobenius norm, the 1-norm (largest column sum), the infinity-norm (largest row sum) and the spectral 2-norm.

$$\|A\|_F = \sqrt{\sum a_{ij}^2},\ \|A\|_1 = \max_j \sum_i |a_{ij}|,\ \|A\|_\infty = \max_i \sum_j |a_{ij}|,\ \|A\|_2 = \sigma_{\max}$$

symbol		meaning	SI unit
`A`	in	the matrix	—
`frobenius`	out	Frobenius norm	—
`norm1`	out	1-norm: largest absolute column sum	—
`norm_inf`	out	infinity-norm: largest absolute row sum	—
`norm2`	out	spectral norm: the largest singular value	—

Applies when: at most 10 x 10.

```
$ linear-algebra solve matrix_norms "A=[[1, -2], [3, 4]]"
frobenius = 5.477225575
norm1 = 6
norm_inf = 7
norm2 = 5.116672736
  by: Norms of a matrix  (exact sums for the Frobenius, 1- and infinity-norms; one-sided Jacobi SVD
for the 2-norm)
  assumes: at most 10 x 10
```

What the worked example means: This calculation determines four different measures of magnitude for the matrix A. The Frobenius norm, equal to 5.47723, represents the square root of the sum of all squared entries, effectively treating the matrix as a vector in Euclidean space. The 1-norm, shown as 6, identifies the largest absolute column sum, while the infinity-norm, given by 7, finds the largest absolute row sum. Finally, the spectral 2-norm, reported as 5.11667, corresponds to the largest singular value of the matrix, indicating the maximum stretching factor applied to any unit vector.

Power of a square matrix — `matrix_power`

A^k for a square matrix A and a whole number k (A^0 = I).

$$A^k = \underbrace{AA\cdots A}_{k}$$

symbol		meaning	SI unit
`A`	in	a square matrix (at most 6 x 6)	—
`k`	in	the exponent (0 to 64)	—
`power`	out	the matrix A^k	—

Applies when: A is square; k is a whole number, 0 <= k <= 64.

```
$ linear-algebra solve matrix_power "A=[[1, 1], [0, 1]]" k=3
power = [[1.0,3.0],[0.0,1.0]]
  by: Power of a square matrix  (binary powering (repeated squaring) in exact decimal arithmetic)
  assumes: A is square; k is a whole number, 0 <= k <= 64
```

What the worked example means: This calculation determines the result of multiplying a square matrix by itself a specific number of times. Given the input matrix [1 1; 0 1] and the whole number exponent 3, the algorithm uses repeated squaring to efficiently compute the final matrix [[1, 3], [0, 1]]. This operation transforms the original data structure into its powered form, representing the cumulative effect of applying the linear transformation defined by the matrix repeatedly.

Product of two matrices — `matrix_multiplication`

The matrix product AB (rows of A times columns of B).

$$(AB)_{ij} = \sum_{k} a_{ik} b_{kj}$$

symbol		meaning	SI unit
`A`	in	left matrix	—
`B`	in	right matrix	—
`product`	out	the matrix AB	—

Applies when: columns of A = rows of B; at most 10 rows and columns.

```
$ linear-algebra solve matrix_multiplication "A=[[1, 2], [3, 4]]" "B=[[5, 6], [7, 8]]"
product = [[19.0,22.0],[43.0,50.0]]
  by: Product of two matrices  (row-by-column sums in exact decimal arithmetic)
  assumes: columns of A = rows of B; at most 10 rows and columns
```

What the worked example means: This calculation computes the matrix product AB, resulting in [[19, 22], [43, 50]]. It combines the rows of the left matrix [1 2; 3 4] with the columns of the right matrix [5 6; 7 8] using row-by-column sums. The result represents the composition of the linear transformations defined by these matrices, allowing for the analysis of sequential operations in systems.

Rank of a matrix — `matrix_rank`

The rank: the number of linearly independent rows (equivalently columns) of a matrix.

$$\operatorname{rank} A = \dim \operatorname{col} A$$

symbol		meaning	SI unit
`A`	in	the matrix (up to 10 x 10)	—
`rank`	out	rank of A	—

Applies when: at most 10 x 10.

```
$ linear-algebra solve matrix_rank "A=[[1, 2, 3], [2, 4, 6], [1, 0, 1]]"
rank = 2
  by: Rank of a matrix  (count of pivots of the reduced row echelon form, exact rational
arithmetic)
  assumes: at most 10 x 10
```

What the worked example means: This calculation determines the rank of the input matrix A, which is defined as the dimension of its column space. The method involves computing the reduced row echelon form using exact rational arithmetic and counting the pivot positions. The resulting value 2 indicates how many rows or columns are linearly independent, providing insight into the matrix's structural properties such as invertibility or the dimensionality of its solution space.

Scalar multiple of a matrix — `scalar_matrix_multiplication`

Multiply every entry of a matrix by a number k.

$$(kA)_{ij} = k\,a_{ij}$$

symbol		meaning	SI unit
`k`	in	the scalar	—
`A`	in	the matrix	—
`product`	out	the matrix k A	—

```
$ linear-algebra solve scalar_matrix_multiplication k=3 "A=[[1, 2], [3, 4]]"
product = [[3.0,6.0],[9.0,12.0]]
  by: Scalar multiple of a matrix  (entry-by-entry multiplication (exact decimal arithmetic))
  k = 3
```

What the worked example means: The calculation produces the matrix [[3, 6], [9, 12]] by applying the scalar 3 to each element of the input matrix [1 2; 3 4]. This operation scales the entire array uniformly, meaning every numerical value within the original structure is multiplied by the same factor. The resulting matrix maintains the same dimensions as the input but reflects the magnitude change dictated by the scalar.

Sum of two matrices — `matrix_addition`

Add two matrices of the same shape entry by entry.

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

symbol		meaning	SI unit
`A`	in	first matrix	—
`B`	in	second matrix	—
`sum`	out	the matrix A + B	—

Applies when: A and B have the same shape.

```
$ linear-algebra solve matrix_addition "A=[[1, 2], [3, 4]]" "B=[[5, 6], [7, 8]]"
sum = [[6.0,8.0],[10.0,12.0]]
  by: Sum of two matrices  (entry-by-entry addition (exact decimal arithmetic))
  assumes: A and B have the same shape
```

What the worked example means: This calculation finds the sum [[6, 8], [10, 12]] by performing entry-by-entry addition on two matrices, [1 2; 3 4] and [5 6; 7 8], which must share the same dimensions. Each element in the resulting matrix is the arithmetic sum of the elements at the identical position in the input matrices. This operation effectively combines the data or transformations represented by [1 2; 3 4] and [5 6; 7 8] into a single aggregate matrix.

Trace of a square matrix — `matrix_trace`

The trace: the sum of the diagonal entries of a square matrix.

$$\operatorname{tr} A = \sum_i a_{ii}$$

symbol		meaning	SI unit
`A`	in	a square matrix	—
`trace`	out	tr(A) = sum of a_ii	—

Applies when: A is square.

```
$ linear-algebra solve matrix_trace "A=[[1, 2], [3, 4]]"
trace = 5
  by: Trace of a square matrix  (sum of the diagonal (exact))
  assumes: A is square
```

What the worked example means: The calculation determines the trace, a scalar value representing the sum of the main diagonal elements of the square matrix. Adding these specific entries together gives 5. This result provides a concise summary of the matrix's scale along its principal axes and remains unchanged regardless of how the matrix basis is rotated, making it a fundamental invariant in linear algebra.

Transpose of a matrix — `matrix_transpose`

Swap rows and columns: the transpose A^T with (A^T)_ij = a_ji.

$$(A^T)_{ij} = a_{ji}$$

symbol		meaning	SI unit
`A`	in	the matrix	—
`transpose`	out	the matrix A^T	—

```
$ linear-algebra solve matrix_transpose "A=[[1, 2, 3], [4, 5, 6]]"
transpose = [[1.0,4.0],[2.0,5.0],[3.0,6.0]]
  by: Transpose of a matrix  (row i of A becomes column i)
```

What the worked example means: This calculation produces the transpose [[1, 4], [2, 5], [3, 6]], which is derived from the original matrix [1 2 3; 4 5 6]. The process involves converting every row in [1 2 3; 4 5 6] into a corresponding column in [[1, 4], [2, 5], [3, 6]], effectively reflecting the elements across the main diagonal. Consequently, the element located at position i, j in the result corresponds to the element at position j, i in the source matrix.

Matrix functions (2)

Matrix exponential — `matrix_exponential`

exp(A) = I + A + A^2/2! + ..., the solution operator of x' = A x.

$$e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!} = \left(e^{A/2^s}\right)^{2^s}$$

symbol		meaning	SI unit
`A`	in	a square matrix with |A|_1 <= 50	—

symbol		meaning	SI unit
`exponential`	out	the matrix exp(A)	—

Applies when: A is square, at most 10 x 10; |A|_1 <= 50.

```
$ linear-algebra solve matrix_exponential "A=[[0, 1], [0, 0]]"
exponential = [[1.0,1.0],[0.0,1.0]]
  by: Matrix exponential  (scaling and squaring: a Taylor series for exp(A / 2^s) with |A / 2^s|_1
<= 1/2, then s squarings)
  assumes: A is square, at most 10 x 10; |A|_1 <= 50
```

What the worked example means: This calculation computes the matrix exponential, denoted as e to the power of A, for the square matrix [0 1; 0 0]. The result, [[1, 1], [0, 1]], represents the fundamental solution operator for systems of linear ordinary differential equations with constant coefficients. It describes how a state vector evolves over time under the influence of the dynamics defined by the input matrix.

Square root of a positive definite matrix — `matrix_square_root`

The unique symmetric positive definite X with X^2 = A, for a symmetric positive definite A.

$$A^{1/2} = Q\,\Lambda^{1/2} Q^{T}$$

symbol		meaning	SI unit
`A`	in	a symmetric positive definite matrix	—
`sqrt`	out	the principal square root A^(1/2)	—

Applies when: A is symmetric positive definite; at most 10 x 10.

```
$ linear-algebra solve matrix_square_root "A=[[5, 4], [4, 5]]"
sqrt = [[1.9999999999999991,0.9999999999999996],[0.9999999999999996,1.9999999999999991]]
  by: Square root of a positive definite matrix  (spectral decomposition by Jacobi rotations:
A^(1/2) = Q diag(sqrt(lambda)) Q^T)
  assumes: A is symmetric positive definite; at most 10 x 10
```

What the worked example means: This process calculates the unique symmetric positive definite matrix that, when multiplied by itself, yields the input matrix. The result, denoted as [[2, 1], [1, 2]], represents the square root of [5 4; 4 5]. This operation relies on decomposing the input into its eigenvalues and eigenvectors, taking the square root of each eigenvalue, and then reconstructing the matrix. It ensures that the output is well-defined and possesses the same symmetry properties as the original input.

Numerical PDE (2)

1-D Poisson equation by finite differences — `poisson_1d`

Solve -u'' = f on (0, L) with u(0) = u(L) = 0 on n equally spaced interior points (second-order finite differences).

$$\frac{-u_{k-1} + 2u_k - u_{k+1}}{h^2} = f_k, \quad u_0 = u_{n+1} = 0$$

symbol		meaning	SI unit
`f`	in	values of f at the n interior grid points x_k = k L/(n+1)	—
`L`	in	length of the interval	—
`u`	out	approximate u at the interior points	—
`h`	out	grid spacing L/(n+1)	—

Applies when: L > 0; u = 0 at both ends (homogeneous Dirichlet conditions); at most 200 interior points.

```
$ linear-algebra solve poisson_1d "f=[1, 1, 1]" L=1
u = [0.09375,0.125,0.09375]
h = 0.25
  by: 1-D Poisson equation by finite differences  (the tridiagonal system (-u_(k-1) + 2u_k -
u_(k+1))/h^2 = f_k solved exactly by the Thomas algorithm)
  L = 1
  assumes: L > 0; u = 0 at both ends (homogeneous Dirichlet conditions); at most 200 interior
points
```

What the worked example means: This calculation solves the one-dimensional Poisson equation using finite differences. It computes the approximate solution at interior points, yielding a grid spacing of 0.25 and solution values [0.09375, 0.125, 0.09375]. These results represent the steady-state distribution of the quantity u across the interval defined by length 1, assuming zero boundary conditions.

2-D Poisson equation on a grid (5-point stencil) — `poisson_2d`

Solve -(u_xx + u_yy) = f on a rectangle with u = 0 on the boundary, on an n x m grid of interior points with spacing h.

$$\frac{4u_{i,j} - u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1}}{h^2} = f_{i,j}$$

symbol		meaning	SI unit
`f`	in	values of f at the interior grid points (row i, column j)	—
`h`	in	grid spacing (equal in x and y)	—
`u`	out	approximate u at the interior grid points	—

Applies when: h > 0; u = 0 on the boundary; at most 12 x 12 interior points.

```
$ linear-algebra solve poisson_2d "f=[[1, 1], [1, 1]]" h=0.5
u = [[0.125,0.125],[0.12499999999999999,0.12499999999999999]]
  by: 2-D Poisson equation on a grid (5-point stencil)  (the 5-point finite-difference system
(4u_ij - neighbours)/h^2 = f_ij, solved by banded Gaussian elimination)
  h = 0.5
  assumes: h > 0; u = 0 on the boundary; at most 12 x 12 interior points
```

What the worked example means: This calculation determines the approximate solution to the two-dimensional Poisson equation on a rectangular domain with zero boundary conditions. It uses a five-point finite difference stencil to convert the partial differential equation into a system of linear equations, which is then solved using banded Gaussian elimination. The resulting values for the potential field u at each interior

point are given by [[0.125, 0.125], [0.125, 0.125]], derived from the source function values f and the uniform grid spacing h.

Numerical methods (16)

Clamped cubic spline interpolation — `clamped_cubic_spline`

The value at a point of the cubic spline through the data with prescribed slopes at the two ends.

$$2h_0M_0 + h_0M_1 = 6\Big(\frac{y_1 - y_0}{h_0} - s_0\Big),\ h_{n-1}M_{n-1} + 2h_{n-1}M_n = 6\Big(s_n - \frac{y_n - y_{n-1}}{h_{n-1}}\Big)$$

symbol		meaning	SI unit
`x`	in	knots, strictly increasing	—
`y`	in	values at the knots	—
`slope_start`	in	slope S'(x0)	—
`slope_end`	in	slope S'(xn)	—
`at`	in	the point (within the knots)	—
`value`	out	spline value S(at)	—

Applies when: x strictly increasing, at least 2 knots (at most 60); x[0] <= at <= x[-1].

```
$ linear-algebra solve clamped_cubic_spline "x=[0, 1, 2]" "y=[0, 1, 0]" at=0.5 slope_start=1
slope_end=-1
value = 0.625
  by: Clamped cubic spline interpolation  (solve the tridiagonal system with clamped end equations
for the knot second derivatives exactly, then evaluate the cubic piece)
  at = 0.5
  slope_start = 1
  slope_end = -1
  assumes: x strictly increasing, at least 2 knots (at most 60); x[0] <= at <= x[-1]
```

What the worked example means: This calculation determines the height of a smooth curve that passes through your data points while matching the specified slopes at both ends. The method solves for the curvature at each knot using a tridiagonal system, then evaluates the specific cubic segment containing the target location. For the input point 0.5, the resulting interpolated value is 0.625, representing the best estimate based on the provided knots and boundary conditions.

Coefficients of the interpolating polynomial — `interpolating_polynomial`

The polynomial c0 + c1 x + ... + c(n-1) x^(n-1) passing exactly through n data points.

$$\sum_k c_k x_i^k = y_i$$

symbol		meaning	SI unit
`x`	in	x values (distinct)	—
`y`	in	y values	—

symbol		meaning	SI unit
`coefficients`	out	[c0, c1, ...], lowest power first	—

Applies when: distinct x values; at most 15 points.

```
$ linear-algebra solve interpolating_polynomial "x=[0, 1, 2]" "y=[1, 3, 7]"
coefficients = [1.0,1.0,1.0]
  by: Coefficients of the interpolating polynomial  (solve the Vandermonde system V c = y in exact
rational arithmetic)
  assumes: distinct x values; at most 15 points
```

What the worked example means: This calculation determines the coefficients for the unique polynomial that passes exactly through the provided data points. By solving a linear system in exact rational arithmetic, it produces the power basis representation [1, 1, 1] corresponding to the input x values [0, 1, 2] and y values [1, 3, 7]. The result allows for precise symbolic manipulation or evaluation of the interpolant defined by these distinct coordinates.

Composite midpoint rule for a polynomial — `midpoint_rule`

Approximate the integral of a polynomial over [a, b] by the composite midpoint rule with n subintervals.

$$\int_a^b p \approx h \sum_{k=0}^{n-1} p\big(a + (k + \tfrac{1}{2})h\big)$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first (written out in x in a question)	—
`a`	in	lower limit	—
`b`	in	upper limit	—
`n`	in	number of subintervals	—
`integral`	out	midpoint-rule estimate	—

Applies when: a < b; 1 <= n <= 1000.

```
$ linear-algebra solve midpoint_rule "coefficients=[1, 0, 0]" a=0 b=2 n=4
integral = 2.625
  by: Composite midpoint rule for a polynomial  (h sum p(a + (k + 1/2) h), h = (b - a)/n, exact
decimal arithmetic)
  a = 0
  b = 2
  assumes: a < b; 1 <= n <= 1000
```

What the worked example means: This calculation approximates the definite integral of a polynomial over the interval from 0 to 2 using the composite midpoint rule with 4 subintervals. The method evaluates the polynomial at the center of each subinterval and sums these values multiplied by the width of the subintervals, yielding an estimated area under the curve of 2.625. This approach balances errors on either side of the midpoint, often providing better accuracy than endpoint rules for smooth functions.

Derivative of equally spaced data (finite differences) — `numerical_derivative`

Estimate dy/dx at every sample of equally spaced data: central differences inside, one-sided differences at the ends.

$$y'_i \approx \frac{y_{i+1} - y_{i-1}}{2h}$$

symbol		meaning	SI unit
`y`	in	samples y_0, ..., y_n (at least 2)	—
`h`	in	spacing between samples	—
`derivative`	out	estimated dy/dx at each sample	—

Applies when: h > 0; at least 2 samples (at most 200).

```
$ linear-algebra solve numerical_derivative "y=[0, 1, 4, 9, 16]" h=1
derivative = [1.0,2.0,4.0,6.0,7.0]
  by: Derivative of equally spaced data (finite differences)  ((y_(i+1) - y_(i-1))/(2h) inside;
(y_1 - y_0)/h and (y_n - y_(n-1))/h at the ends (exact decimal arithmetic))
  h = 1
  assumes: h > 0; at least 2 samples (at most 200)
```

What the worked example means: This calculation estimates the rate of change for equally spaced data points. Using the central difference formula for interior points and one-sided differences at the boundaries, it processes the input samples with a fixed spacing of 1. The result is an array of estimated derivatives, representing how quickly the values are changing at each specific location in the dataset.

Evaluate a polynomial and its derivative (Horner's method) — `polynomial_evaluation`

The value p(x) and the derivative p'(x) of a polynomial at a point.

$$p(x) = (\cdots((a_n x + a_{n-1})x + a_{n-2})x + \cdots)x + a_0$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first (written out in x in a question)	—
`x`	in	the point	—
`value`	out	p(x)	—
`derivative`	out	p'(x)	—

Applies when: at most degree 12.

```
$ linear-algebra solve polynomial_evaluation "coefficients=[2, -6, 2, -1]" x=3
value = 5
derivative = 20
  by: Evaluate a polynomial and its derivative (Horner's method)  (Horner's scheme with the
derivative carried along (exact decimal arithmetic))
  x = 3
  assumes: at most degree 12
```

What the worked example means: This calculation determines the value of a polynomial and its first derivative at a specific point using Horner's method. Given coefficients in descending order of powers and an input point x, the algorithm efficiently computes p(x) as 5 and p'(x) as 20. The result represents the function's height and slope at that location, assuming exact decimal arithmetic and a degree no greater than twelve.

Gauss-Legendre quadrature nodes and weights — `gauss_legendre_rule`

The nodes and weights of the n-point Gauss-Legendre rule on [-1, 1] (exact for polynomials of degree up to 2n - 1).

$$\int_{-1}^{1} f(x)\,dx \approx \sum_{i=1}^{n} w_i f(x_i), \quad P_n(x_i) = 0$$

symbol		meaning	SI unit
`n`	in	number of points (1 to 40)	—
`nodes`	out	nodes x_i in [-1, 1], ascending	—
`weights`	out	weights w_i	—

Applies when: 1 <= n <= 40.

```
$ linear-algebra solve gauss_legendre_rule n=3
nodes = [-0.7745966692414834,0.0,0.7745966692414834]
weights = [0.5555555555555552,0.8888888888888888,0.5555555555555552]
  by: Gauss-Legendre quadrature nodes and weights  (Newton's method on the Legendre polynomial P_n
(three-term recurrence); w_i = 2 / ((1 - x_i^2) P_n'(x_i)^2))
  assumes: 1 <= n <= 40
```

What the worked example means: This calculation determines the specific points and weights required for Gauss-Legendre quadrature on the interval from negative one to positive one. Given an input of 3 points, the algorithm finds the roots of the Legendre polynomial to produce the ascending nodes [-0.774597, 0, 0.774597] and calculates the corresponding integration coefficients as [0.555556, 0.888889, 0.555556]. These values allow for exact integration of polynomials up to degree 3 doubled minus one.

Gauss-Legendre quadrature of a polynomial — `gauss_legendre_integral`

The n-point Gauss-Legendre approximation of the integral of a polynomial over [a, b] (exact up to degree 2n - 1).

$$\int_{a}^{b} p(x)\,dx \approx \frac{b-a}{2} \sum_{i} w_i\, p\Big(\frac{a+b}{2} + \frac{b-a}{2} x_i\Big)$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first (written out in x in a question)	—
`a`	in	lower limit	—
`b`	in	upper limit	—
`n`	in	number of Gauss points (1 to 40)	—

symbol		meaning	SI unit
`integral`	out	Gauss-Legendre estimate	—

Applies when: a < b; 1 <= n <= 40.

```
$ linear-algebra solve gauss_legendre_integral "coefficients=[1, 1, 0, 0]" a=0 b=2 n=2
integral = 6.666666667
  by: Gauss-Legendre quadrature of a polynomial  (map the n Gauss-Legendre nodes to [a, b]: (b -
a)/2 sum w_i p((a + b)/2 + (b - a)/2 x_i))
  a = 0
  b = 2
  assumes: a < b; 1 <= n <= 40
```

What the worked example means: This calculation determines the approximate definite integral of a polynomial using Gauss-Legendre quadrature. The method maps standard nodes and weights to the specified interval defined by the lower limit 0 and upper limit 2, applying 2 evaluation points. The resulting estimate for the area under the curve is 6.66667. This approach provides high accuracy for polynomials up to degree two times the number of points minus one, effectively capturing the behavior of the function without requiring excessive subdivisions.

Lagrange interpolation — `lagrange_interpolation`

The value at a point of the polynomial of lowest degree through given data points (Lagrange form).

$$p(t) = \sum_i y_i \prod_{j \neq i} \frac{t - x_j}{x_i - x_j}$$

symbol		meaning	SI unit
`x`	in	x values of the data (distinct)	—
`y`	in	y values of the data	—
`at`	in	the point where the interpolant is evaluated	—
`value`	out	interpolated value p(at)	—

Applies when: distinct x values; x and y of equal length (at most 60).

```
$ linear-algebra solve lagrange_interpolation "x=[0, 1, 2]" "y=[1, 3, 2]" at=1.5
value = 2.875
  by: Lagrange interpolation  (the Lagrange formula sum y_i prod (t - x_j)/(x_i - x_j), exact
decimal arithmetic)
  at = 1.5
  assumes: distinct x values; x and y of equal length (at most 60)
```

What the worked example means: This calculation determines the value of the unique polynomial that passes through all provided data points. By using the Lagrange interpolation formula, it computes the exact decimal result for the input point 1.5, yielding the output 2.875. This method constructs a curve of the lowest possible degree that fits the given x and y coordinates without solving for individual coefficients.

Natural cubic spline interpolation — `natural_cubic_spline`

The value at a point of the natural cubic spline through the data (second derivative zero at both ends).

$$h_{i-1}M_{i-1} + 2(h_{i-1}+h_i)M_i + h_iM_{i+1} = 6\left(\frac{y_{i+1}-y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right),\ M_0 = M_n = 0$$

symbol		meaning	SI unit
`x`	in	knots, strictly increasing (at least 3)	—
`y`	in	values at the knots	—
`at`	in	the point (within the knots)	—
`value`	out	spline value S(at)	—

Applies when: x strictly increasing, at least 3 knots (at most 60); x[0] <= at <= x[-1]; natural end conditions S''(x0) = S''(xn) = 0.

```
$ linear-algebra solve natural_cubic_spline "x=[0, 1, 2]" "y=[0, 1, 0]" at=0.5
value = 0.6875
  by: Natural cubic spline interpolation  (solve the tridiagonal system for the knot second
derivatives exactly, then evaluate the cubic piece)
  at = 0.5
  assumes: x strictly increasing, at least 3 knots (at most 60); x[0] <= at <= x[-1]; natural end
conditions S''(x0) = S''(xn) = 0
```

What the worked example means: This calculation determines the smooth curve passing through your data points using a natural cubic spline. It solves for the second derivatives at each knot to ensure continuity, then evaluates the specific cubic segment containing the target point. The resulting value, 0.6875, represents the interpolated height of this smooth curve at the position 0.5, assuming zero curvature at both ends of the dataset.

Newton divided differences — `newton_divided_differences`

The coefficients f[x0], f[x0,x1], ..., f[x0..xn] of the Newton form of the interpolating polynomial.

$$f[x_i,\ldots,x_{i+k}] = \frac{f[x_{i+1},\ldots,x_{i+k}] - f[x_i,\ldots,x_{i+k-1}]}{x_{i+k} - x_i}$$

symbol		meaning	SI unit
`x`	in	x values (distinct)	—
`y`	in	y values	—
`coefficients`	out	divided differences f[x0], f[x0,x1], ...: p(t) = c0 + c1 (t - x0) + c2 (t - x0)(t - x1) + ...	—

Applies when: distinct x values; x and y of equal length (at most 60).

```
$ linear-algebra solve newton_divided_differences "x=[1, 2, 4]" "y=[1, 4, 16]"
coefficients = [1.0,3.0,1.0]
  by: Newton divided differences  (the divided-difference table, exact decimal arithmetic)
  assumes: distinct x values; x and y of equal length (at most 60)
```

What the worked example means: This calculation constructs the divided difference table to determine the coefficients for the Newton form of the interpolating polynomial. Using the distinct x values [1, 2, 4] and corresponding y values [1, 4, 16], it computes the sequence [1, 3, 1]. These coefficients allow the polynomial to be written as a sum where each term multiplies by factors like t minus x, ensuring the curve passes exactly through all provided data points while maintaining numerical stability for incremental updates.

Piecewise linear interpolation — `linear_interpolation`

The value at a point of the broken line joining consecutive data points.

$$y = y_k + (y_{k+1} - y_k)\frac{t - x_k}{x_{k+1} - x_k}$$

symbol		meaning	SI unit
`x`	in	x values, strictly increasing	—
`y`	in	y values	—
`at`	in	the point (within the data range)	—
`value`	out	interpolated value	—

Applies when: x strictly increasing; x[0] <= at <= x[-1] (no extrapolation).

```
$ linear-algebra solve linear_interpolation "x=[0, 1, 3]" "y=[0, 2, 3]" at=2
value = 2.5
  by: Piecewise linear interpolation  (locate the interval containing the point and interpolate
linearly (exact))
  at = 2
  assumes: x strictly increasing; x[0] <= at <= x[-1] (no extrapolation)
```

What the worked example means: Piecewise linear interpolation calculates the value on the broken line connecting known data points. For the input point 2, which lies within the range of strictly increasing x values, the method identifies the surrounding interval and applies a linear formula. The resulting output is 2.5, representing the estimated position on that straight-line segment between the neighboring data points.

Root of a polynomial by bisection — `bisection_root`

The root of a polynomial inside an interval [a, b] where it changes sign, by repeated halving.

$$m = \tfrac{a+b}{2};\ [a,b] \leftarrow [a,m] \text{ if } p(a)p(m) < 0 \text{ else } [m,b]$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first (written out in x in a question)	—
`a`	in	left end of the interval	—
`b`	in	right end of the interval	—
`root`	out	the root in [a, b]	—

Applies when: a < b; p(a) and p(b) have opposite signs; exactly one root in (a, b) (checked exactly by a Sturm sequence); the root is simple and well conditioned.

```
$ linear-algebra solve bisection_root "coefficients=[1, 0, -2]" a=1 b=2
root = 1.414213562
  by: Root of a polynomial by bisection  (bisection until the interval cannot be halved in floating point (about 52 halvings per unit))
  a = 1
  b = 2
  assumes: a < b; p(a) and p(b) have opposite signs; exactly one root in (a, b) (checked exactly by a Sturm sequence); the root is simple and well conditioned
```

What the worked example means: This calculation determines the specific value where the polynomial equals zero, located strictly between the left endpoint 1 and the right endpoint 2. The method repeatedly halves this interval, selecting the sub-interval that contains a sign change, until the precision limit of floating-point arithmetic is reached. The resulting value 1.41421 represents the unique simple root guaranteed to exist in this range by the Intermediate Value Theorem.

Root of a polynomial by Newton's method — `newton_root`

A root of a polynomial by Newton's method x <- x - p(x)/p'(x) from a starting value x0.

$$x_{k+1} = x_k - \frac{p(x_k)}{p'(x_k)}$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first (written out in x in a question)	—
`x0`	in	starting value	—
`root`	out	the root the iteration converges to	—

Applies when: the iteration converges (within 200 steps) to a simple, well-conditioned root; p'(x) never vanishes at an iterate.

```
$ linear-algebra solve newton_root "coefficients=[1, 0, -2, -5]" x0=2
root = 2.094551482
  by: Root of a polynomial by Newton's method  (Newton-Raphson iteration until the step is below the last bit of x)
  x0 = 2
  assumes: the iteration converges (within 200 steps) to a simple, well-conditioned root; p'(x) never vanishes at an iterate
```

What the worked example means: This calculation determines a root of the polynomial defined by the coefficients [1, 0, -2, -5] using Newton-Raphson iteration. Starting from the initial guess 2, the method repeatedly updates the estimate by subtracting the ratio of the polynomial's value to its derivative until convergence. The final result, 2.09455, represents the x-value where the polynomial equals zero, assuming the conditions for simple roots and non-vanishing derivatives are met.

Root of a polynomial by the secant method — `secant_root`

A root of a polynomial by the secant method from two starting values x0 and x1.

$$x_{k+1} = x_k - p(x_k)\,\frac{x_k - x_{k-1}}{p(x_k) - p(x_{k-1})}$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first (written out in x in a question)	—
`x0`	in	first starting value	—
`x1`	in	second starting value	—
`root`	out	the root the iteration converges to	—

Applies when: x0 != x1; the iteration converges (within 200 steps) to a simple, well-conditioned root.

```
$ linear-algebra solve secant_root "coefficients=[1, 0, -2, -5]" x0=2 x1=3
root = 2.094551482
  by: Root of a polynomial by the secant method  (secant iteration x_(k+1) = x_k - p(x_k)(x_k -
x_(k-1))/(p(x_k) - p(x_(k-1))))
  x0 = 2
  x1 = 3
  assumes: x0 != x1; the iteration converges (within 200 steps) to a simple, well-conditioned root
```

What the worked example means: This calculation determines a root of the polynomial defined by coefficients [1, 0, -2, -5] using the secant method. The algorithm iteratively approximates the zero crossing by drawing a secant line between two previous estimates, starting with initial values 2 and 3. After convergence within the specified step limit, the resulting root is reported as 2.09455. This value represents the point where the polynomial evaluates to zero, found without requiring explicit derivative calculations.

Simpson's rule for tabulated data — `simpson_rule`

The integral of equally spaced tabulated data by the composite Simpson 1/3 rule.

$$\int y\,dx \approx \frac{h}{3}\left(y_0 + 4\sum_{\text{odd}} y_i + 2\sum_{\text{even}} y_i + y_n\right)$$

symbol		meaning	SI unit
`x`	in	equally spaced, increasing x values (odd count)	—
`y`	in	y values	—
`integral`	out	Simpson estimate of the integral	—

Applies when: x equally spaced and increasing; an odd number of points, at least 3 (an even number of intervals).

```
$ linear-algebra solve simpson_rule "x=[0, 1, 2]" "y=[1, 2, 5]"
integral = 4.666666667
  by: Simpson's rule for tabulated data  (h/3 (y_0 + 4 y_1 + 2 y_2 + 4 y_3 + ... + y_n), exact
decimal arithmetic)
  assumes: x equally spaced and increasing; an odd number of points, at least 3 (an even number of
intervals)
```

What the worked example means: This calculation applies Simpson's one-third rule to estimate the definite integral of a function represented by discrete data points. By assuming the x values are equally spaced and there is an odd number of them, the method fits parabolic arcs through consecutive triplets of points to approximate the area under the curve more accurately than simple trapezoidal methods for smooth functions. The resulting value, 4.66667, represents this numerical approximation of the total accumulated quantity defined by the y values over the range of x.

Trapezoidal rule for tabulated data — `trapezoid_rule`

The integral of tabulated data by the composite trapezoidal rule (any spacing).

$$\int y\,dx \approx \sum_i \frac{x_{i+1} - x_i}{2}(y_i + y_{i+1})$$

symbol		meaning	SI unit
`x`	in	x values, strictly increasing	—
`y`	in	y values	—
`integral`	out	trapezoidal estimate of the integral	—

Applies when: x strictly increasing; at least 2 points (at most 60).

```
$ linear-algebra solve trapezoid_rule "x=[0, 1, 2, 4]" "y=[1, 3, 2, 0]"
integral = 6.5
  by: Trapezoidal rule for tabulated data  (sum of (x_(i+1) - x_i)(y_i + y_(i+1))/2, exact decimal
arithmetic)
  assumes: x strictly increasing; at least 2 points (at most 60)
```

What the worked example means: This calculation determines the approximate area under a curve defined by discrete data points using the composite trapezoidal rule. It sums the areas of individual trapezoids formed between consecutive pairs of x and y values, where each width is multiplied by the average height of its two endpoints. The resulting value, 6.5, represents the total estimated integral for the provided dataset, accommodating irregular spacing between the x coordinates.

Sparse matrices (4)

Compressed sparse column (CSC) form of a matrix — `csc_format`

Store a matrix in compressed sparse column form: the nonzero values, their row indices and the column pointers.

$$\mathtt{data}[\mathtt{indptr}[k] : \mathtt{indptr}[k+1]]$$

symbol		meaning	SI unit
`A`	in	the matrix (up to 50 x 50)	—
`data`	out	nonzero values, column by column	—
`indices`	out	0-based row index of each value	—
`indptr`	out	column pointers: the values of column k are data[indptr[k]:indptr[k+1]]	—

symbol		meaning	SI unit
`nnz`	out	number of stored nonzeros	—
`density`	out	nnz / (rows x columns)	—

Applies when: at most 50 x 50; zeros are not stored; indices sorted within each column (canonical form).

```
$ linear-algebra solve csc_format "A=[[1, 0, 2], [0, 0, 3], [4, 5, 0]]"
data = [1.0,4.0,5.0,2.0,3.0]
indices = [0,2,2,0,1]
indptr = [0,2,3,5]
nnz = 5
density = 0.5555555556
  by: Compressed sparse column (CSC) form of a matrix  (scan column by column, keep the nonzeros)
  assumes: at most 50 x 50; zeros are not stored; indices sorted within each column (canonical
form)
```

What the worked example means: The algorithm scans the input matrix [1 0 2; 0 0 3; 4 5 0] column by column to identify nonzero entries. It produces three arrays: data containing the nonzero values, indices storing their zero-based row locations, and indptr marking the start of each column. The total count of stored elements is 5, resulting in a sparsity density of 0.555556. This structure allows efficient memory usage for matrices that contain mostly zeros.

Compressed sparse row (CSR) form of a matrix — `csr_format`

Store a matrix in compressed sparse row form: the nonzero values, their column indices and the row pointers.

$$\mathtt{data}[\mathtt{indptr}[k] : \mathtt{indptr}[k+1]]$$

symbol		meaning	SI unit
`A`	in	the matrix (up to 50 x 50)	—
`data`	out	nonzero values, row by row	—
`indices`	out	0-based column index of each value	—
`indptr`	out	row pointers: the values of row k are data[indptr[k]:indptr[k+1]]	—
`nnz`	out	number of stored nonzeros	—
`density`	out	nnz / (rows x columns)	—

Applies when: at most 50 x 50; zeros are not stored; indices sorted within each row (canonical form).

```
$ linear-algebra solve csr_format "A=[[1, 0, 2], [0, 0, 3], [4, 5, 0]]"
data = [1.0,2.0,3.0,4.0,5.0]
indices = [0,2,2,0,1]
indptr = [0,2,3,5]
nnz = 5
density = 0.5555555556
  by: Compressed sparse row (CSR) form of a matrix  (scan row by row, keep the nonzeros)
  assumes: at most 50 x 50; zeros are not stored; indices sorted within each row (canonical form)
```

What the worked example means: This computation converts the input matrix A into the compressed sparse row format, which efficiently stores only the non-zero elements. The result includes an array of values [1, 2, 3,

4, 5] ordered by row, their corresponding zero-based column indices [0, 2, 2, 0, 1], and pointer offsets [0, 2, 3, 5] that mark where each row begins in the storage arrays. The total number of stored entries is given by 5, while the sparsity level is quantified by the density ratio 0.555556. This structure allows algorithms to skip zero entries during operations like matrix-vector multiplication.

Dense matrix from CSR arrays — `csr_to_dense`

Rebuild the full matrix from compressed sparse row arrays (data, indices, indptr) and the number of columns.

$$A_{r,\,\mathtt{indices}[k]} \mathrel{+}= \mathtt{data}[k],\ \mathtt{indptr}[r] \le k < \mathtt{indptr}[r+1]$$

symbol		meaning	SI unit
`data`	in	stored values	—
`indices`	in	0-based column index of each value	—
`indptr`	in	row pointers (rows + 1 entries)	—
`ncols`	in	number of columns	—
`dense`	out	the full matrix	—

Applies when: indptr starts at 0, never decreases, ends at len(data); indices within 0..ncols-1; duplicate entries are summed; at most 50 x 50.

```
$ linear-algebra solve csr_to_dense "data=[1, 2, 3]" "indices=[0, 2, 1]" "indptr=[0, 2, 3]" ncols=3
dense = [[1.0,0.0,2.0],[0.0,3.0,0.0]]
  by: Dense matrix from CSR arrays  (row r holds data[indptr[r]:indptr[r+1]] at columns
indices[indptr[r]:indptr[r+1]])
  assumes: indptr starts at 0, never decreases, ends at len(data); indices within 0..ncols-1;
duplicate entries are summed; at most 50 x 50
```

What the worked example means: This calculation rebuilds a full matrix, shown as [[1, 0, 2], [0, 3, 0]], by placing each value from [1, 2, 3] into its corresponding row and column specified by [0, 2, 3] and [0, 2, 1]. The number of columns in the final result is determined by 3. This process converts compressed storage back into a standard grid format for easier inspection or further processing.

Sparse matrix times vector (CSR) — `sparse_matvec`

The product y = A x for a matrix A given in compressed sparse row form.

$$y_r = \sum_{k=\mathtt{indptr}[r]}^{\mathtt{indptr}[r+1]-1} \mathtt{data}[k]\, x_{\mathtt{indices}[k]}$$

symbol		meaning	SI unit
`data`	in	stored values	—
`indices`	in	0-based column index of each value	—
`indptr`	in	row pointers (rows + 1 entries)	—
`x`	in	the vector (one entry per column)	—
`product`	out	the vector A x	—

Applies when: consistent CSR arrays; column indices below len(x); at most 50 x 50.

```
$ linear-algebra solve sparse_matvec "data=[1, 2, 3]" "indices=[0, 2, 1]" "indptr=[0, 2, 3]" "x=[1,
1, 1]"
product = [3.0,3.0]
  by: Sparse matrix times vector (CSR)  (y_r = sum over the stored entries of row r of data[k]
x[indices[k]] (exact))
  assumes: consistent CSR arrays; column indices below len(x); at most 50 x 50
```

What the worked example means: This calculation determines the result of multiplying a sparse matrix by a vector using Compressed Sparse Row format. It processes only the stored non-zero values, specifically combining the entries in [1, 2, 3] with corresponding elements from [1, 1, 1] as indicated by [0, 2, 1] and [0, 2, 3]. The final output is the resulting vector [3, 3], which represents the linear transformation applied to the input vector while efficiently skipping zero elements to reduce computational overhead.

Vector spaces (5)

Column space of a matrix — `column_space`

A basis of the column space (range) of A: the pivot columns of A itself, and its dimension.

$$\operatorname{col} A = \operatorname{span}\{\mathbf{a}_j : j \text{ a pivot column}\}$$

symbol		meaning	SI unit
`A`	in	the matrix (up to 10 x 10)	—
`dimension`	out	dimension of the column space (the rank)	—
`basis`	out	the pivot columns of A, left to right	—

Applies when: at most 10 x 10.

```
$ linear-algebra solve column_space "A=[[1, 2, 3], [2, 4, 7]]"
dimension = 2
basis = [[1.0,2.0],[3.0,7.0]]
  by: Column space of a matrix  (the pivot positions of the exact reduced row echelon form pick the
columns of the ORIGINAL matrix)
  assumes: at most 10 x 10
```

What the worked example means: This calculation determines a basis for the column space of matrix A by identifying the pivot positions in its reduced row echelon form. The resulting basis, [[1, 2], [3, 7]], comprises the corresponding columns taken directly from the original matrix A, while the rank or dimension of this subspace is reported as 2. This set of vectors spans all possible linear combinations of the columns of A, defining the range of the associated linear transformation.

Coordinates of a vector in a basis — `change_of_basis`

The coordinates c of v with respect to a basis b1, ..., bn: v = c1 b1 + ... + cn bn.

$$\mathbf{v} = \sum_i c_i \mathbf{b}_i$$

symbol		meaning	SI unit
`basis`	in	the basis vectors, one per row	—
`v`	in	the vector	—
`coordinates`	out	the coordinate vector [v]_B	—

Applies when: n basis vectors of length n that are linearly independent; v has n entries.

```
$ linear-algebra solve change_of_basis "basis=[[1, 1], [1, -1]]" "v=[3, 1]"
coordinates = [2.0,1.0]
  by: Coordinates of a vector in a basis  (solve B c = v with the basis vectors as the columns of B
(exact))
  assumes: n basis vectors of length n that are linearly independent; v has n entries
```

What the worked example means: This calculation determines the specific weights needed to express the input vector as a linear combination of the basis vectors. Solving the system where the basis vectors form the columns of a matrix shows that the vector equals [2, 1]. These values represent the position of the vector within the coordinate system defined by the basis, allowing for a different perspective on the same geometric object.

Gram-Schmidt orthonormalization — `gram_schmidt`

Turn linearly independent vectors v1, ..., vk into orthonormal vectors e1, ..., ek spanning the same nested subspaces.

$$\mathbf{w}_j = \mathbf{v}_j - \sum_{i<j} (\mathbf{e}_i \cdot \mathbf{v}_j)\,\mathbf{e}_i, \quad \mathbf{e}_j = \mathbf{w}_j / \|\mathbf{w}_j\|$$

symbol		meaning	SI unit
`vectors`	in	the vectors, one per row (independent)	—
`orthonormal`	out	orthonormal vectors e1, ..., ek, one per row	—

Applies when: the vectors are linearly independent; at most 10 vectors of at most 10 entries.

```
$ linear-algebra solve gram_schmidt "vectors=[[3, 4], [1, 0]]"
orthonormal = [[0.6,0.8],[0.7999999999999999,-0.6]]
  by: Gram-Schmidt orthonormalization  (modified Gram-Schmidt with re-orthogonalization (e_j . v_j
> 0 by construction))
  assumes: the vectors are linearly independent; at most 10 vectors of at most 10 entries
```

What the worked example means: This calculation applies the modified Gram-Schmidt process to transform a set of linearly independent vectors into an orthonormal basis. Given the input vectors [3 4; 1 0], the algorithm subtracts projections onto previously computed orthogonal directions and normalizes the result. The output is the set of orthonormal vectors [[0.6, 0.8], [0.8, -0.6]], which span the same nested subspaces as the original inputs while ensuring each vector has unit length and is perpendicular to all others.

Linear independence of vectors — `linear_independence`

Whether a list of vectors is linearly independent, with the rank of the list.

$$c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0} \Rightarrow c_1 = \cdots = c_k = 0$$

symbol		meaning	SI unit
`vectors`	in	the vectors, one per row (equal lengths)	—
`independent`	out	true when no vector is a combination of the others	—
`rank`	out	dimension of their span	—

Applies when: all vectors have the same length; at most 10 vectors of at most 10 entries.

```
$ linear-algebra solve linear_independence "vectors=[[1, 2, 3], [4, 5, 6], [7, 8, 9]]"
independent = false
rank = 2
  by: Linear independence of vectors  (the vectors are independent exactly when the rank of the
matrix of rows equals their number (exact))
  assumes: all vectors have the same length; at most 10 vectors of at most 10 entries
```

What the worked example means: This calculation determines whether the provided list of vectors is linearly independent by checking if the only solution to the equation where their weighted sum equals zero is when all weights are zero. The result shows that the vectors are no, and the dimension of the space they span, known as the rank, is 2. This means no vector in the set can be expressed as a combination of the others.

Null space (kernel) of a matrix — `null_space`

The dimension of the null space {x : Ax = 0} (the nullity) and a basis of it.

$$\operatorname{null} A = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}, \quad \dim = n - \operatorname{rank} A$$

symbol		meaning	SI unit
`A`	in	the matrix (up to 10 x 10)	—
`nullity`	out	dimension of the null space = columns - rank	—
`basis`	out	basis vectors: one per free variable, that variable 1, the other free variables 0	—

Applies when: at most 10 x 10.

```
$ linear-algebra solve null_space "A=[[1, 2, 3], [2, 4, 6]]"
nullity = 2
basis = [[-2.0,1.0,0.0],[-3.0,0.0,1.0]]
  by: Null space (kernel) of a matrix  (solve Ax = 0 from the exact reduced row echelon form; each
free variable gives one basis vector)
  assumes: at most 10 x 10
```

What the worked example means: This calculation determines the null space of matrix A, which consists of all vectors x such that Ax equals the zero vector. The result provides the dimension of this space, known as the nullity, reported here as 2, and a set of basis vectors, shown as [[-2, 1, 0], [-3, 0, 1]]. These vectors form the fundamental solutions to the homogeneous system derived from the reduced row echelon form of the input matrix.

Vectors (13)

Angle between two vectors — `angle_between_vectors`

The angle between two nonzero vectors, in degrees and radians (between 0 and 180 degrees).

$$\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}$$

symbol		meaning	SI unit
`u`	in	first vector	—
`v`	in	second vector	—
`angle_degrees`	out	angle in degrees	—
`angle_radians`	out	angle in radians	—

Applies when: u and v have the same number of entries; neither vector is zero.

```
$ linear-algebra solve angle_between_vectors "u=[1, 0]" "v=[1, 1]"
angle_degrees = 45
angle_radians = 0.7853981634
  by: Angle between two vectors  (theta = 2 atan2(|a - b|, |a + b|) with a, b the unit vectors
(accurate also near 0 and 180 degrees))
  assumes: u and v have the same number of entries; neither vector is zero
```

What the worked example means: This calculation finds the geometric angle separating two nonzero vectors in space. It uses the dot product of vector u and vector v divided by the product of their magnitudes to determine the cosine of the angle. The result, expressed as 45 degrees, indicates how much the directions of the two vectors differ, ranging from zero when they point in the same direction to one hundred eighty degrees when they are opposite.

Area of the parallelogram (and triangle) spanned by two vectors — `parallelogram_area`

|u x v|: the area of the parallelogram with sides u and v (2-D or 3-D), and half of it for the triangle.

$$A = \|\mathbf{u}\times\mathbf{v}\| = \sqrt{\|\mathbf{u}\|^2\|\mathbf{v}\|^2 - (\mathbf{u}\cdot\mathbf{v})^2}$$

symbol		meaning	SI unit
`u`	in	first side (2 or 3 entries)	—
`v`	in	second side (2 or 3 entries)	—
`area`	out	parallelogram area |u x v|	—
`triangle_area`	out	triangle area |u x v| / 2	—

Applies when: u and v have the same number of entries, 2 or 3.

```
$ linear-algebra solve parallelogram_area "u=[3, 0]" "v=[1, 2]"
area = 6
triangle_area = 3
  by: Area of the parallelogram (and triangle) spanned by two vectors  (Lagrange's identity |u x
v|^2 = |u|^2 |v|^2 - (u . v)^2 (exact under the root))
  assumes: u and v have the same number of entries, 2 or 3
```

What the worked example means: This calculation determines the geometric area enclosed by two vectors, [3, 0] and [1, 2]. The primary result, 6, represents the area of the parallelogram formed by these sides. Since a triangle with these same sides covers exactly half of that parallelogram, the value 3 provides the corresponding triangular area. These measurements are derived using Lagrange's identity, which relates the squared magnitude of the cross product to the dot product and individual magnitudes, ensuring an exact result under the square root.

Cross product of two 3-D vectors — `cross_product`

The cross product u x v of two vectors in three dimensions (right-hand rule).

$$\mathbf{u} \times \mathbf{v} = \left(u_2 v_3 - u_3 v_2,\ u_3 v_1 - u_1 v_3,\ u_1 v_2 - u_2 v_1\right)$$

symbol		meaning	SI unit
`u`	in	first vector (3 entries)	—
`v`	in	second vector (3 entries)	—
`cross`	out	the vector u x v	—

Applies when: u and v have exactly 3 entries.

```
$ linear-algebra solve cross_product "u=[1, 2, 3]" "v=[4, 5, 6]"
cross = [-3.0,6.0,-3.0]
  by: Cross product of two 3-D vectors  (determinant formula (u2 v3 - u3 v2, u3 v1 - u1 v3, u1 v2 -
u2 v1), exact decimal arithmetic)
  assumes: u and v have exactly 3 entries
```

What the worked example means: This calculation determines the cross product, which yields a new vector perpendicular to both input vectors in three-dimensional space. The result [-3, 6, -3] is derived from the components of the first vector [1, 2, 3] and the second vector [4, 5, 6] using the determinant formula. This output represents the normal vector to the plane containing the two inputs, following the right-hand rule convention.

Difference of two vectors — `vector_subtraction`

Subtract the second vector from the first, entry by entry.

$$\mathbf{u} - \mathbf{v} = \left(u_1 - v_1, \ldots, u_n - v_n\right)$$

symbol		meaning	SI unit
`u`	in	first vector	—
`v`	in	vector subtracted	—

symbol		meaning	SI unit
`difference`	out	the vector u - v	—

Applies when: u and v have the same number of entries.

```
$ linear-algebra solve vector_subtraction "u=[5, 3]" "v=[2, 7]"
difference = [3.0,-4.0]
  by: Difference of two vectors  (entry-by-entry subtraction (exact decimal arithmetic))
  assumes: u and v have the same number of entries
```

What the worked example means: This calculation finds the vector representing the displacement from the second input to the first. It takes the first vector [5, 3] and the second vector [2, 7], then performs entry-by-entry subtraction to produce the result [3, -4]. This output represents the relative position or change between the two original vectors, assuming both have identical dimensions.

Distance between two points — `distance_between_points`

The Euclidean distance between two points given by their coordinates (any dimension).

$$d = \sqrt{\sum_i (p_i - r_i)^2}$$

symbol		meaning	SI unit
`p`	in	coordinates of the first point	—
`r`	in	coordinates of the second point	—
`distance`	out	Euclidean distance |p - r|	—

Applies when: the points have the same number of coordinates.

```
$ linear-algebra solve distance_between_points "p=[1, 2]" "r=[4, 6]"
distance = 5
  by: Distance between two points  (square root of the sum of squared coordinate differences (the
sum exact))
  assumes: the points have the same number of coordinates
```

What the worked example means: This calculation determines the straight-line separation between two points in space. It uses the coordinates of the first point, [1, 2], and the second point, [4, 6]. The method involves summing the squares of the differences between corresponding coordinates and then taking the square root of that sum. The resulting value, 5, represents the geometric length of the line segment connecting these two specific locations.

Dot product of two vectors — `dot_product`

The dot (scalar, inner) product u . v = sum of u_i v_i.

$$\mathbf{u} \cdot \mathbf{v} = \sum_i u_i v_i$$

symbol		meaning	SI unit
`u`	in	first vector	—
`v`	in	second vector	—
`dot`	out	the dot product u . v	—

Applies when: u and v have the same number of entries.

```
$ linear-algebra solve dot_product "u=[1, 2, 3]" "v=[4, -5, 6]"
dot = 12
  by: Dot product of two vectors  (sum of entry-wise products (exact decimal arithmetic))
  assumes: u and v have the same number of entries
```

What the worked example means: This calculation finds the dot product by multiplying corresponding entries of vector u and vector v, then summing those products. The result, 12, represents a single scalar value derived from the two input vectors. This operation measures how much one vector extends in the direction of another, which is fundamental for determining angles between vectors and checking if they are orthogonal.

Length (norm) of a vector — `vector_norm`

The Euclidean length |v| of a vector, with its 1-norm (sum of magnitudes) and max-norm.

$$\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}$$

symbol		meaning	SI unit
`v`	in	the vector	—
`norm`	out	Euclidean length |v|_2	—
`norm1`	out	1-norm: sum of |v_i|	—
`norm_inf`	out	max-norm: largest |v_i|	—

```
$ linear-algebra solve vector_norm "v=[3, 4]"
norm = 5
norm1 = 7
norm_inf = 4
  by: Length (norm) of a vector  (square root of the sum of squares (the sum exact))
```

What the worked example means: This calculation determines the size of a vector using three different measures. The Euclidean norm, representing the straight-line distance from the origin, is found to be 5. The one-norm, which sums the absolute values of each component, equals 7. Finally, the max-norm, defined as the largest absolute value among the components, is 4. These results describe the vector's magnitude in standard geometric terms and alternative metric spaces.

Outer product of two vectors — `outer_product`

The matrix u v^T with entries u_i v_j.

$$(\mathbf{u}\mathbf{v}^T)_{ij} = u_i v_j$$

symbol		meaning	SI unit
`u`	in	first vector (rows)	—
`v`	in	second vector (columns)	—
`outer`	out	the matrix u v^T	—

```
$ linear-algebra solve outer_product "u=[1, 2]" "v=[3, 4, 5]"
outer = [[3.0,4.0,5.0],[6.0,8.0,10.0]]
  by: Outer product of two vectors  (entry (i, j) = u_i v_j (exact))
```

What the worked example means: This calculation constructs a matrix by multiplying every element of the first vector with every corresponding element of the second. Given the input vectors [1, 2] and [3, 4, 5], the resulting matrix [[3, 4, 5], [6, 8, 10]] has entries where each position is the product of one component from the row vector and one from the column vector, forming a rank-one structure.

Projection of one vector onto another — `vector_projection`

The orthogonal projection of u onto v, (u.v / v.v) v, and the scalar projection (component) u.v / |v|.

$$\text{proj}_{\mathbf{v}}\mathbf{u} = \frac{\mathbf{u}\cdot\mathbf{v}}{\mathbf{v}\cdot\mathbf{v}}\,\mathbf{v}$$

symbol		meaning	SI unit
`u`	in	vector to project	—
`v`	in	direction projected onto (not zero)	—
`projection`	out	vector projection of u onto v	—
`scalar_projection`	out	scalar projection u.v / |v|	—

Applies when: u and v have the same number of entries; v is not the zero vector.

```
$ linear-algebra solve vector_projection "u=[2, 3]" "v=[1, 0]"
projection = [2.0,0.0]
scalar_projection = 2
  by: Projection of one vector onto another  (proj = (u.v / v.v) v in exact decimal arithmetic)
  assumes: u and v have the same number of entries; v is not the zero vector
```

What the worked example means: This calculation determines the orthogonal projection of vector u onto vector v. Taking the dot product of u and v, dividing by the dot product of v with itself, and scaling v by that ratio isolates the part of u that points in the direction of v. The resulting vector [2, 0] represents this geometric shadow, while 2 indicates the magnitude of u along the line defined by v.

Scalar multiple of a vector — `scalar_vector_multiplication`

Multiply every entry of a vector by a number k.

$$k\mathbf{v} = (kv_1, \ldots, kv_n)$$

symbol		meaning	SI unit
`k`	in	the scalar	—
`v`	in	the vector	—
`product`	out	the vector k v	—

```
$ linear-algebra solve scalar_vector_multiplication k=4 "v=[1, -2, 3]"
product = [4.0,-8.0,12.0]
  by: Scalar multiple of a vector  (entry-by-entry multiplication (exact decimal arithmetic))
  k = 4
```

What the worked example means: This calculation determines the scalar multiple of a vector by multiplying each individual component of the vector [1, -2, 3] by the single number 4. The output is a new vector, denoted as [4, -8, 12], where every entry has been scaled according to this factor. This operation effectively stretches or compresses the original vector along its existing line of action, changing its magnitude without altering the underlying direction defined by the ratio of its components.

Scalar triple product (volume of a parallelepiped) — `triple_scalar_product`

u . (v x w), the signed volume of the parallelepiped spanned by three 3-D vectors, and its absolute value.

$$\mathbf{u} \cdot (\mathbf{v} \times \mathbf{w}) = \det[\mathbf{u}; \mathbf{v}; \mathbf{w}]$$

symbol		meaning	SI unit
`u`	in	first vector (3 entries)	—
`v`	in	second vector (3 entries)	—
`w`	in	third vector (3 entries)	—
`triple`	out	u . (v x w)	—
`volume`	out	volume |u . (v x w)|	—

Applies when: each vector has exactly 3 entries.

```
$ linear-algebra solve triple_scalar_product "u=[1, 0, 0]" "v=[0, 1, 0]" "w=[0, 0, 1]"
triple = 1
volume = 1
  by: Scalar triple product (volume of a parallelepiped)  (the determinant of the matrix with rows
u, v, w (exact))
  assumes: each vector has exactly 3 entries
```

What the worked example means: This calculation determines the signed volume of the parallelepiped spanned by three vectors in three-dimensional space. Computing the dot product of the first vector with the cross product of the second and third gives the value 1. The absolute magnitude of this result represents the actual geometric volume, which is 1. This metric is essential for understanding spatial relationships and determining if the vectors are coplanar.

Sum of two vectors — `vector_addition`

Add two vectors of the same length entry by entry.

$$\mathbf{u} + \mathbf{v} = (u_1 + v_1, \ldots, u_n + v_n)$$

symbol		meaning	SI unit
`u`	in	first vector	—
`v`	in	second vector	—
`sum`	out	the vector u + v	—

Applies when: u and v have the same number of entries.

```
$ linear-algebra solve vector_addition "u=[1, 2, 3]" "v=[4, 5, 6]"
sum = [5.0,7.0,9.0]
  by: Sum of two vectors  (entry-by-entry addition (exact decimal arithmetic))
  assumes: u and v have the same number of entries
```

What the worked example means: This calculation determines the vector sum by adding the corresponding entries of the two input vectors. Given the first vector [1, 2, 3] and the second vector [4, 5, 6], the result is a new vector [5, 7, 9] where each component is the exact decimal sum of the matching components from the inputs. This operation effectively combines the magnitudes and directions represented by the individual vectors into a single resultant vector.

Unit vector in the direction of a vector — `unit_vector`

Normalize a nonzero vector: v / |v|, the vector of length 1 pointing the same way.

$$\hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\|$$

symbol		meaning	SI unit
`v`	in	the vector (not all zero)	—
`unit`	out	the unit vector v / |v|	—

Applies when: v is not the zero vector.

```
$ linear-algebra solve unit_vector "v=[3, 4]"
unit = [0.6,0.8]
  by: Unit vector in the direction of a vector  (divide each entry by the Euclidean length)
  assumes: v is not the zero vector
```

What the worked example means: To find the unit vector, each component of the input vector [3, 4] is divided by its Euclidean length. The resulting vector [0.6, 0.8] has a magnitude of exactly one but maintains the original orientation of [3, 4]. This normalization process is essential when only the direction of a vector matters, such as in defining axes or calculating projections.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.